mc_libs Comprehensive Library Guide

Generated on 2026-03-04 15:44:50

Contents

1 Packages Overview **5**

2 Library Tree **5**

3 Package `mc_cli` **9**
- 3.1 Module `mc_cli` . . . 9
  - 3.1.1 Function `main` . . . 9

4 Package `mc_data_utils` **9**
- 4.1 Module `mc_data_utils` . . . 9
- 4.2 Module `mc_data_utils.allan` . . . 10
  - 4.2.1 Function `allan2` . . . 10
- 4.3 Module `mc_data_utils.allan_metrics` . . . 10
  - 4.3.1 Function `extract_allan_metrics` . . . 10
- 4.4 Module `mc_data_utils.metrics` . . . 11
  - 4.4.1 Function `first_local_min` . . . 11
  - 4.4.2 Function `nearest_index` . . . 12
- 4.5 Module `mc_data_utils.signal` . . . 12
  - 4.5.1 Function `estimate_period` . . . 12
- 4.6 Module `mc_data_utils.time` . . . 13
  - 4.6.1 Function `time_window_mask_from_hm` . . . 13
  - 4.6.2 Function `time_window_mask_from_indexes` . . . 14
  - 4.6.3 Function `time_window_mask_from_seconds` . . . 14

5 Package `mc_geomap_utils` **15**
- 5.1 Module `mc_geomap_utils` . . . 15
- 5.2 Module `mc_geomap_utils.evaluation` . . . 15
  - 5.2.1 Function `array_stats` . . . 15
  - 5.2.2 Function `print_array_stats` . . . 16
  - 5.2.3 Function `export_label_table` . . . 16
  - 5.2.4 Function `export_feature_stats_table` . . . 16
- 5.3 Module `mc_geomap_utils.io` . . . 17
  - 5.3.1 Function `load_points` . . . 17
  - 5.3.2 Function `save_points` . . . 18
  - 5.3.3 Function `load_grd` . . . 18
  - 5.3.4 Function `write_geotiff` . . . 19

5.3.5 Class PointFileMode . . . 19
5.4 Module mc_geomap_utils.macroregions . . . 19
5.4.1 Function eroded_support_mask . . . 19
5.4.2 Function classify_policy_cells . . . 20
5.4.3 Function grad_mag_deg . . . 21
5.4.4 Function edge_list_with_boundaries . . . 21
5.4.5 Function superpixel_stats . . . 22
5.4.6 Function rag_merge_slic . . . 22
5.4.7 Function axial_mean_heading . . . 23
5.4.8 Function policy_for_class . . . 24
5.4.9 Function policy_superpixel_consensus . . . 24
5.4.10 Function policy_mode_filter . . . 25
5.4.11 Function summarize_regions . . . 25
5.4.12 Function export_regions_csv . . . 26
5.4.13 Function absorb_small_islands . . . 27
5.4.14 Function smooth_macro_labels . . . 27
5.4.15 Class UF . . . 28
5.5 Module mc_geomap_utils.manipulation . . . 28
5.5.1 Function shift_point_cloud . . . 29
5.5.2 Function grid_to_point_cloud . . . 29
5.5.3 Class Mode . . . 30
5.6 Module mc_geomap_utils.planning . . . 30
5.6.1 Function region_mask . . . 30
5.6.2 Function generate_serpentine_tracks_polygon . . . 30
5.6.3 Function generate_serpentine_tracks . . . 31
5.6.4 Function attach_altitude_and_tilt . . . 32
5.6.5 Function estimate_turn_count . . . 33
5.6.6 Function polyline_length_m . . . 33
5.6.7 Function tracks_total_length_m . . . 33
5.6.8 Function merge_tracks_continuous . . . 34
5.6.9 Function straighten_tracks . . . 34
5.6.10 Function smooth_polyline . . . 35
5.6.11 Function simplify_polyline_rdp . . . 35
5.7 Module mc_geomap_utils.plotting . . . 36
5.7.1 Function show_plot . . . 36
5.7.2 Function save_mosaic . . . 36
5.7.3 Function save_mosaic_1x2 . . . 37
5.7.4 Function plot_point_cloud . . . 37
5.7.5 Function plot_mesh . . . 38
5.7.6 Function plot_dem . . . 38
5.7.7 Function plot_feature_map . . . 39
5.7.8 Function plot_labels . . . 40
5.7.9 Function plot_policy_map . . . 40
5.7.10 Function plot_macro_overlay . . . 41
5.7.11 Function plot_macro_with_seeds . . . 41
5.7.12 Function plot_region_headings . . . 42
5.7.13 Function plot_dem_with_policy_overlay . . . 43
5.7.14 Function make_quicklook_figures . . . 44

5.7.15 Function plot_tracks_overlay . . . 45
5.7.16 Function plot_tracks_overlay_by_region . . . 46
5.7.17 Function plot_macro_tracks_overlay . . . 46
5.7.18 Function plot_dem_with_polygon . . . 47
5.7.19 Function plot_primitives_overlay . . . 48
5.7.20 Function plot_knoll_diagnostics . . . 49
5.7.21 Function plot_primitives_3d . . . 50
5.7.22 Function plot_primitives_3d_interactive . . . 50
5.7.23 Function plot_tracks_3d_interactive . . . 51
5.7.24 Function plot_macro_labels_3d_interactive . . . 52
5.7.25 Function plot_knoll_ellipses_3d . . . 53
5.7.26 Class PlotDataSource . . . 53
5.8 Module mc_geomap_utils.pointcloud . . . 54
5.8.1 Function delaunay_mesh . . . 54
5.8.2 Function mesh_to_obj . . . 55
5.8.3 Function delaunay_to_open3d_mesh . . . 55
5.8.4 Function grid_to_dem . . . 56
5.8.5 Function dem_to_features . . . 56
5.8.6 Function feature_to_slic_segmentation . . . 57
5.9 Module mc_geomap_utils.primitives . . . 58
5.9.1 Function detect_knolls . . . 58
5.9.2 Function detect_knolls_plane_descent . . . 59
5.9.3 Function augment_regions_with_primitives . . . 60
5.9.4 Class Knoll . . . 61
5.10 Module mc_geomap_utils.spiral . . . 61
5.10.1 Function archimedean_spiral . . . 61
5.10.2 Function clip_polyline_to_mask . . . 62

6 Package mc_io_utils **63**
6.1 Module mc_io_utils . . . 63
6.2 Module mc_io_utils.adapters . . . 63
6.2.1 Function from_imu_kearfott_compas . . . 63
6.3 Module mc_io_utils.io . . . 64
6.3.1 Function load_csv_timeseries_to_mc_kinematics . . . 64
6.3.2 Function load_points . . . 65
6.3.3 Function list_files . . . 65
6.3.4 Function save_points . . . 66
6.3.5 Function load_grd . . . 66
6.3.6 Function write_geotiff . . . 67
6.3.7 Class PointFileMode . . . 67
6.4 Module mc_io_utils.lcm . . . 68
6.4.1 Function scan_channels . . . 68
6.4.2 Function scan_logs_detailed . . . 68
6.4.3 Function list_channel_fields . . . 69
6.4.4 Function list_channel_fields_in_dir . . . 70
6.4.5 Function lcmlog_to_pickle . . . 70
6.4.6 Function main . . . 71
6.4.7 Class ChannelTimingStats . . . 71

6.4.8 Class SyncAgeStats . . . 72
6.4.9 Class FileScanStats . . . 72
6.4.10 Class LCMScanStats . . . 73
6.5 Module mc_io_utils.runtime . . . 73
6.5.1 Function clear_console . . . 73
6.5.2 Function setup_logging_from_yaml . . . 73
6.5.3 Function check_output_file . . . 74
6.5.4 Function sum_total_bytes . . . 74
6.5.5 Function estimate_total_bytes . . . 75

7 Package mc_libs_index **76**
7.1 Module mc_libs_index . . . 76
7.1.1 Function list_cli_commands . . . 76
7.2 Module mc_libs_index.index . . . 76
7.2.1 Function discover_libraries . . . 76
7.2.2 Function build_registry . . . 77
7.2.3 Function list_libraries . . . 77
7.2.4 Function list_functions . . . 78
7.2.5 Function list_examples . . . 78
7.2.6 Function list_all . . . 78
7.2.7 Function list_all_auto . . . 79
7.2.8 Function get_help . . . 79
7.2.9 Function get_example . . . 80
7.2.10 Function tree . . . 80
7.2.11 Function print_tree . . . 81
7.2.12 Function list_cli_commands . . . 81

8 Package mc_reporting_utils **82**
8.1 Module mc_reporting_utils . . . 82
8.2 Module mc_reporting_utils.formatting . . . 82
8.2.1 Function format_bytes_human . . . 82
8.2.2 Function format_timestamp_utc_us . . . 82
8.2.3 Function format_duration_us . . . 83
8.2.4 Function format_mean_ms . . . 83
8.3 Module mc_reporting_utils.terminal . . . 84
8.3.1 Class Spinner . . . 84

9 Package mc_robo_utils **85**
9.1 Module mc_robo_utils . . . 85
9.2 Module mc_robo_utils.kinematics . . . 85
9.2.1 Function build_component . . . 85
9.2.2 Function default_units . . . 86
9.2.3 Class KinematicComponent . . . 86
9.2.4 Class KinematicFrame . . . 87
9.2.5 Class KinematicsData . . . 87
9.3 Module mc_robo_utils.plotting . . . 87
9.3.1 Function plot_component . . . 87
9.3.2 Function plot_multiple . . . 88

9.3.3 Function `plot_series` 89
9.3.4 Function `plot_multi_series` 89
9.3.5 Function `plot_allan` 90
9.3.6 Function `save_html` 91
9.4 Module `mc_robo_utils.plotting_print` 92
9.4.1 Function `plot_multiple_mpl` 92
9.4.2 Function `plot_series_mpl` 92
9.4.3 Function `plot_multi_series_mpl` 93
9.4.4 Function `plot_allan_mpl` 93
9.5 Module `mc_robo_utils.transformations` 94
9.5.1 Function `quat_to_rpy` 94

This document is generated from source modules and docstrings under the mc_libs repository.

1 Packages Overview

- mc_cli
- mc_data_utils
- mc_geomap_utils
- mc_io_utils
- mc_libs_index
- mc_reporting_utils
- mc_robo_utils

2 Library Tree

```
mc_libs
|- mc_cli
| `- mc_cli
| `- main
|- mc_data_utils
| |- mc_data_utils
| |- mc_data_utils.allan
| | `- allan2
| |- mc_data_utils.allan_metrics
| | `- extract_allan_metrics
| |- mc_data_utils.metrics
| | |- first_local_min
| | `- nearest_index
| |- mc_data_utils.signal
| | `- estimate_period
| `- mc_data_utils.time
| |- time_window_mask_from_hm
| |- time_window_mask_from_indexes
| `- time_window_mask_from_seconds
|- mc_geomap_utils
| |- mc_geomap_utils
| |- mc_geomap_utils.evaluation
| | |- array_stats
| | |- export_feature_stats_table
| | |- export_label_table
| | `- print_array_stats
| |- mc_geomap_utils.io
| | |- PointFileMode
| | |- load_grd
```

```
| | |- load_points
| | |- save_points
| | `- write_geotiff
| |- mc_geomap_utils.macroregions
| | |- UF.find
| | |- UF.union
| | |- absorb_small_islands
| | |- axial_mean_heading
| | |- classify_policy_cells
| | |- edge_list_with_boundaries
| | |- eroded_support_mask
| | |- export_regions_csv
| | |- grad_mag_deg
| | |- policy_for_class
| | |- policy_mode_filter
| | |- policy_superpixel_consensus
| | |- rag_merge_slic
| | |- smooth_macro_labels
| | |- summarize_regions
| | `- superpixel_stats
| |- mc_geomap_utils.manipulation
| | |- Mode
| | |- grid_to_point_cloud
| | `- shift_point_cloud
| |- mc_geomap_utils.planning
| | |- attach_altitude_and_tilt
| | |- estimate_turn_count
| | |- generate_serpentine_tracks
| | |- generate_serpentine_tracks_polygon
| | |- merge_tracks_continuous
| | |- polyline_length_m
| | |- region_mask
| | |- simplify_polyline_rdp
| | |- smooth_polyline
| | |- straighten_tracks
| | `- tracks_total_length_m
| |- mc_geomap_utils.plotting
| | |- PlotDataSource.from_input
| | |- PlotDataSource.iter_xy_pairs
| | |- make_quicklook_figures
| | |- plot_dem
| | |- plot_dem_with_policy_overlay
| | |- plot_dem_with_polygon
| | |- plot_feature_map
| | |- plot_knoll_diagnostics
| | |- plot_knoll_ellipses_3d
| | |- plot_labels
| | |- plot_macro_labels_3d_interactive
```

```
| | |- plot_macro_overlay
| | |- plot_macro_tracks_overlay
| | |- plot_macro_with_seeds
| | |- plot_mesh
| | |- plot_point_cloud
| | |- plot_policy_map
| | |- plot_primitives_3d
| | |- plot_primitives_3d_interactive
| | |- plot_primitives_overlay
| | |- plot_region_headings
| | |- plot_tracks_3d_interactive
| | |- plot_tracks_overlay
| | |- plot_tracks_overlay_by_region
| | |- save_mosaic
| | |- save_mosaic_1x2
| | `- show_plot
| |- mc_geomap_utils.pointcloud
| | |- delaunay_mesh
| | |- delaunay_to_open3d_mesh
| | |- dem_to_features
| | |- feature_to_slic_segmentation
| | |- grid_to_dem
| | `- mesh_to_obj
| |- mc_geomap_utils.primitives
| | |- Knoll
| | |- augment_regions_with_primitives
| | |- detect_knolls
| | `- detect_knolls_plane_descent
| `- mc_geomap_utils.spiral
| |- archimedean_spiral
| `- clip_polyline_to_mask
|- mc_io_utils
| |- mc_io_utils
| |- mc_io_utils.adapters
| | `- from_imu_kearfott_compas
| |- mc_io_utils.io
| | |- PointFileMode
| | |- list_files
| | |- load_csv_timeseries_to_mc_kinematics
| | |- load_grd
| | |- load_points
| | |- save_points
| | `- write_geotiff
| |- mc_io_utils.lcm
| | |- ChannelTimingStats.add_event
| | |- FileScanStats
| | |- LCMScanStats
| | |- SyncAgeStats.observe
```

```
| | |- lcmlog_to_pickle
| | |- list_channel_fields
| | |- list_channel_fields_in_dir
| | |- main
| | |- scan_channels
| | `- scan_logs_detailed
| `- mc_io_utils.runtime
| |- check_output_file
| |- clear_console
| |- estimate_total_bytes
| |- setup_logging_from_yaml
| `- sum_total_bytes
|- mc_libs_index
| |- mc_libs_index
| | `- list_cli_commands
| `- mc_libs_index.index
| |- build_registry
| |- discover_libraries
| |- get_example
| |- get_help
| |- list_all
| |- list_all_auto
| |- list_cli_commands
| |- list_examples
| |- list_functions
| |- list_libraries
| |- print_tree
| `- tree
|- mc_reporting_utils
| |- mc_reporting_utils
| |- mc_reporting_utils.formatting
| | |- format_bytes_human
| | |- format_duration_us
| | |- format_mean_ms
| | `- format_timestamp_utc_us
| `- mc_reporting_utils.terminal
| |- Spinner.start
| `- Spinner.stop
`- mc_robo_utils
|- mc_robo_utils
|- mc_robo_utils.kinematics
| |- KinematicComponent.as_dict
| |- KinematicFrame
| |- KinematicsData.copy
| |- build_component
| `- default_units
|- mc_robo_utils.plotting
| |- plot_allan
```

```
| |- plot_component
| |- plot_multi_series
| |- plot_multiple
| |- plot_series
| `- save_html
|- mc_robo_utils.plotting_print
| |- plot_allan_mpl
| |- plot_multi_series_mpl
| |- plot_multiple_mpl
| `- plot_series_mpl
`- mc_robo_utils.transformations
`- quat_to_rpy
```

3 Package `mc_cli`

3.1 Module `mc_cli`

CLI for mc_* libraries.

3.1.1 Function `main`

Entry point for the "mc-libs" CLI.

Syntax

```
from mc_cli import main
main() -> int
```

Inputs

```
None
```

Outputs

```
int: exit code (0 for success).
```

Raises Not specified.

Example

```
>>> from mc_cli import main
>>> isinstance(main(), int)
True
```

4 Package `mc_data_utils`

4.1 Module `mc_data_utils`

General data utilities.

Exports: - allan2 - estimate_period - first_local_min, nearest_index - extract_allan_metrics - time_window_mask_from_hm, time_window_mask_from_indexes, time_window_mask_from_seconds

No public functions or classes were found in this module.

4.2 Module `mc_data_utils.allan`

Allan variance/deviation utilities.
Functions: - allan2(omega, fs, pts)

4.2.1 Function `allan2`

Compute Allan deviation (sigma) using log-spaced cluster sizes.

Syntax

```
from mc_data_utils.allan import allan2
allan2(omega: np.ndarray, fs: float, pts: int) -> Tuple[np.ndarray, np.ndarray]
```

Inputs

```
omega (np.ndarray): Rate sequence shaped ``(N,)`` or ``(N, M)``, where ``M`` is the
    number of axes.
fs (float): Sample rate in Hz.
pts (int): Number of log-spaced cluster sizes (tau points) to evaluate.
```

Outputs

```
Tuple[np.ndarray, np.ndarray]: ``T`` (tau values in seconds) and ``sigma`` with
shape ``(len(T), M)`` containing Allan deviation per axis.
```

Raises

```
ValueError: If fewer than 3 samples are provided.
```

Example

```
>>> import numpy as np
>>> from mc_data_utils.allan import allan2
>>> omega = np.random.randn(1000)
>>> T, sigma = allan2(omega, fs=100.0, pts=50)
```

4.3 Module `mc_data_utils.allan_metrics`

Allan deviation metric extraction helpers.
Functions: - extract_allan_metrics(T, sigma, ...)

4.3.1 Function `extract_allan_metrics`

Extract Allan-derived metrics per axis.

Syntax

```
from mc_data_utils.allan_metrics import extract_allan_metrics
extract_allan_metrics(
    T: np.ndarray,
    sigma: np.ndarray,
    *,
    kind: str = "gyro_deg_per_hr",
    min_points: int = 8,
    slope_tol: float = 0.12,
```

```
    bias_slope_tol: float = 0.08,
    bias_divisor: float = 0.664,
) -> Dict[str, Dict[str, float]]
```

Inputs

```
T (np.ndarray): averaging times (tau) in seconds, shape ``(K,)``.
sigma (np.ndarray): Allan deviation array, shape ``(K, M)`` (or ``(K,)``).
    Units must match the input to ``allan2``. For example:
    - ``kind="gyro_deg_per_hr"`` expects ``sigma`` in deg/hr.
    - ``kind="accel_mg"`` expects ``sigma`` in mG.
kind (str): conversion convention. Supported values:
    - ``"gyro_deg_per_hr"`` -> ARW [deg/sqrthr], RRW [deg/hr/sqrthr], BS [deg/hr]
    - ``"accel_mg"`` -> VRW [mG/sqrthr], BRW [mG/sqrthr], BS [mG]
    The input Allan curve is fit in tau seconds; coefficients are converted
    to per-``sqrt(hr)`` style metrics commonly reported in datasheets.
min_points (int): minimum number of points for slope segment fits.
slope_tol (float): tolerance for slope matching when finding white/RW segments.
bias_slope_tol (float): reserved tolerance for bias estimation (kept for API
    compatibility).
bias_divisor (float): divisor for bias instability extraction (default 0.664).
```

Outputs

```
Dict[str, Dict[str, float]]: metrics per axis key (``"X"``, ``"Y"``, ``"Z"``). Each
axis dict includes ``white_coeff``, ``rw_coeff``, ``bs``, and diagnostic tau ranges.
```

Raises

```
ValueError: If ``T`` and ``sigma`` lengths are inconsistent.
```

Example

```
>>> import numpy as np
>>> from mc_data_utils.allan_metrics import extract_allan_metrics
>>> T = np.logspace(0, 3, 50)
>>> sigma = np.random.rand(50, 3)
>>> metrics = extract_allan_metrics(T, sigma, kind="gyro_deg_per_hr")
```

4.4 Module `mc_data_utils.metrics`

Small metric helpers.

Functions: - first_local_min(x) - nearest_index(x, target)

4.4.1 Function `first_local_min`

Return index of first local minimum; fallback to last index.

Syntax

```
from mc_data_utils.metrics import first_local_min
first_local_min(x: np.ndarray) -> int
```

Inputs

```
x (np.ndarray): 1-D array to scan.
```

Outputs

```
int: index of the first local minimum, or the last index if none found.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_data_utils.metrics import first_local_min
>>> first_local_min(np.array([3.0, 2.0, 4.0, 1.0]))
1
```

4.4.2 Function `nearest_index`

Return index of element in x closest to target.

Syntax

```
from mc_data_utils.metrics import nearest_index
nearest_index(x: np.ndarray, target: float) -> int
```

Inputs

```
x (np.ndarray): 1-D array of candidate values.
target (float): target value to match.
```

Outputs

```
int: index of the nearest value in ``x``.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_data_utils.metrics import nearest_index
>>> nearest_index(np.array([0.0, 2.0, 5.0]), 3.1)
2
```

4.5 Module `mc_data_utils.signal`

Signal analysis utilities.

Functions: - estimate_period(t, y)

4.5.1 Function `estimate_period`

Estimate dominant period using FFT; returns period in seconds.

Syntax

```
from mc_data_utils.signal import estimate_period
estimate_period(t: np.ndarray, y: np.ndarray) -> float
```

Inputs

```
t (np.ndarray): time stamps (seconds).
y (np.ndarray): signal samples aligned with ``t``.
```

Outputs

```
float: estimated dominant period in seconds (``nan`` if insufficient data).
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_data_utils.signal import estimate_period
>>> t = np.linspace(0, 10, 500)
>>> y = np.sin(2 * np.pi * t)
>>> round(estimate_period(t, y), 2)
1.0
```

4.6 Module `mc_data_utils.time`

Time-window utilities.

Functions: - time_window_mask_from_hm(t_dt, start_hm=None, end_hm=None) - time_window_mask_f start_idx=None, end_idx=None) - time_window_mask_from_seconds(t_s, start_s=None, end_s=None)

4.6.1 Function `time_window_mask_from_hm`

Return a boolean mask for a HH:MM time window on datetime series.

If end <= start, the window is assumed to wrap into the next day.

Syntax

```
from mc_data_utils.time import time_window_mask_from_hm
time_window_mask_from_hm(
    t_dt: list[datetime], start_hm: str | None, end_hm: str | None
) -> np.ndarray
```

Inputs

```
t_dt (list[datetime]): datetime samples to filter.
start_hm (str | None): start time in ``HH:MM`` format (local time).
end_hm (str | None): end time in ``HH:MM`` format (local time).
```

Outputs

```
np.ndarray: boolean mask aligned to ``t_dt``.
```

Raises Not specified.

Example

```
>>> from datetime import datetime
>>> from mc_data_utils.time import time_window_mask_from_hm
>>> t = [datetime(2024, 1, 1, 8, 0), datetime(2024, 1, 1, 9, 0)]
>>> time_window_mask_from_hm(t, "08:30", "09:30").tolist()
[False, True]
```

4.6.2 Function `time_window_mask_from_indexes`

Return a boolean mask for a slice by index range.

start_idx/end_idx are inclusive bounds. If None, defaults to start/end.

Syntax

```
from mc_data_utils.time import time_window_mask_from_indexes
time_window_mask_from_indexes(
    n: int, start_idx: int | None = None, end_idx: int | None = None
) -> np.ndarray
```

Inputs

```
n (int): length of the sequence to mask.
start_idx (int | None): inclusive start index (default 0).
end_idx (int | None): inclusive end index (default ``n - 1``).
```

Outputs

```
np.ndarray: boolean mask of length ``n``.
```

Raises

```
ValueError: If ``n`` is negative or indices are negative.
```

Example

```
>>> from mc_data_utils.time import time_window_mask_from_indexes
>>> time_window_mask_from_indexes(5, start_idx=1, end_idx=3).tolist()
[False, True, True, True, False]
```

4.6.3 Function `time_window_mask_from_seconds`

Return a boolean mask for a time window based on seconds array.

Syntax

```
from mc_data_utils.time import time_window_mask_from_seconds
time_window_mask_from_seconds(
    t_s: np.ndarray, start_s: float | None = None, end_s: float | None = None
) -> np.ndarray
```

Inputs

```
t_s (np.ndarray): time vector in seconds.
start_s (float | None): start time (defaults to first element).
end_s (float | None): end time (defaults to last element).
```

Outputs

```
np.ndarray: boolean mask aligned to ``t_s``.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_data_utils.time import time_window_mask_from_seconds
>>> t = np.array([0.0, 1.0, 2.0, 3.0])
>>> time_window_mask_from_seconds(t, 1.0, 2.0).tolist()
[False, True, True, False]
```

5 Package `mc_geomap_utils`

5.1 Module `mc_geomap_utils`

Reusable utilities for point-cloud processing and visualization.

No public functions or classes were found in this module.

5.2 Module `mc_geomap_utils.evaluation`

No description provided.

5.2.1 Function `array_stats`

Summarise the finite values of a raster with key statistics.

Syntax

```
from mc_geomap_utils.evaluation import array_stats
array_stats(arr: np.ndarray) -> dict[str, float]
```

Inputs

```
arr (np.ndarray): array from which to compute summary statistics.
```

Outputs

```
dict[str, float]: dictionary containing counts, percentiles, and extrema.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.evaluation import array_stats
>>> stats = array_stats(np.array([1.0, 2.0, np.nan]))
>>> stats["valid_px"]
2.0
```

5.2.2 Function `print_array_stats`

Pretty-print summary statistics produced by “array_stats“.

Syntax

```
from mc_geomap_utils.evaluation import print_array_stats
print_array_stats(arr: np.ndarray, name: str) -> None
```

Inputs

```
arr (np.ndarray): array to analyse.
name (str): label used in the console output.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.evaluation import print_array_stats
>>> print_array_stats(np.array([1.0, 2.0, np.nan]), "demo")
```

5.2.3 Function `export_label_table`

Write CSV with per-label pixel counts and area in square metres.

Syntax

```
from mc_geomap_utils.evaluation import export_label_table
export_label_table(labels: np.ndarray, cell: float, path: Path | str) -> None
```

Inputs

```
labels (np.ndarray): integer label raster to summarise.
cell (float): cell size used to convert pixel counts to area.
path (Path | str): destination CSV file path.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.evaluation import export_label_table
>>> export_label_table(np.array([[0, 1], [1, 2]]), 0.5, "labels.csv")
```

5.2.4 Function `export_feature_stats_table`

Serialise feature statistics to CSV using “array_stats“.

Syntax

```
from mc_geomap_utils.evaluation import export_feature_stats_table
export_feature_stats_table(features: dict[str, np.ndarray], path: Path | str) -> None
```

Inputs

```
features (dict[str, np.ndarray]): mapping of feature name to raster.
path (Path | str): destination CSV file path.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.evaluation import export_feature_stats_table
>>> export_feature_stats_table({"slope": np.array([[1.0, np.nan]])}, "features.csv")
```

5.3 Module `mc_geomap_utils.io`

No description provided.

5.3.1 Function `load_points`

Load a whitespace-delimited point cloud into a NumPy array.

Syntax

```
from mc_geomap_utils.io import load_points
load_points(file_path: Path, *, mode: PointFileMode | str = PointFileMode.VERTICAL) ->
    NDArray[np.float64]
```

Inputs

```
file_path (Path): absolute path to the source XYZ text file.
mode (PointFileMode | str): "vertical" for one point per line, "horizontal" for per-axis
    rows.
```

Outputs

```
NDArray[np.float64]: array with columns ``x, y, z`` describing each point.
```

Raises

```
ValueError: If the file contents are malformed or incompatible with the chosen layout.
FileNotFoundError: If ``file_path`` does not exist.
```

Example

```
>>> from pathlib import Path
>>> from mc_geomap_utils.io import load_points
>>> pts = load_points(Path("points.xyz"))
```

5.3.2 Function `save_points`

Persist a point cloud next to the original file with a “*_cleaned“ suffix.

Syntax

```
from mc_geomap_utils.io import save_points
save_points(file_path: Path, points: np.ndarray) -> Path
```

Inputs

```
file_path (Path): original input file whose directory will host the cleaned copy.
points (np.ndarray): coordinates to be written (shape N x 3).
```

Outputs

```
Path: filesystem path of the generated ``*_cleaned.txt`` file.
```

Raises Not specified.

Example

```
>>> from pathlib import Path
>>> import numpy as np
>>> from mc_geomap_utils.io import save_points
>>> out_path = save_points(Path("points.xyz"), np.zeros((3, 3)))
```

5.3.3 Function `load_grd`

Load a GMT/NetCDF “.grd“ file and return its raster plus metadata.

Syntax

```
from mc_geomap_utils.io import load_grd
load_grd(
    file_path: Path | str,
    *,
    assume_wgs84: bool = True,
) -> tuple[np.ndarray, Any, Any | None]
```

Inputs

```
file_path (Path | str): path to the grid file on disk.
assume_wgs84 (bool): inject ``EPSG:4326`` when the dataset lacks a CRS.
```

Outputs

```
tuple[np.ndarray, Any, Any | None]: data array, affine transform, and CRS (if present).
```

Raises

```
FileNotFoundError: If the grid file does not exist.
RuntimeError: If ``rasterio`` is unavailable.
```

Example

```
>>> from mc_geomap_utils.io import load_grd
>>> data, transform, crs = load_grd("surface.grd")
```

5.3.4 Function `write_geotiff`

Persist a NumPy array as a single-band GeoTIFF (with .npy fallback).

Syntax

```
from mc_geomap_utils.io import write_geotiff
write_geotiff(
    path: Path | str,
    arr: np.ndarray,
    transform: Any,
    *,
    crs: str | None = None,
    nodata: float | None = np.nan,
) -> None
```

Inputs

```
path (Path | str): destination file path for the raster.
arr (np.ndarray): image data to be written.
transform (Any): affine transform describing pixel-to-world mapping.
crs (str | None): coordinate reference system identifier.
nodata (float | None): value representing no-data cells.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.io import write_geotiff
>>> write_geotiff("out.tif", np.zeros((10, 10)), transform=None)
```

5.3.5 Class `PointFileMode`

Layout of coordinates inside a plain-text point cloud file.

No public methods were found.

5.4 Module `mc_geomap_utils.macroregions`

No description provided.

5.4.1 Function `eroded_support_mask`

Erode valid-data support by N cells to hide edge seams.

Syntax

```
from mc_geomap_utils.macroregions import eroded_support_mask
eroded_support_mask(dem: np.ndarray, border: int = 3) -> np.ndarray
```

Inputs

```
dem (np.ndarray): DEM raster with NaNs for no-data.
border (int): number of cells to erode inward.
```

Outputs

```
np.ndarray: boolean mask of the eroded valid-data support.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import eroded_support_mask
>>> mask = eroded_support_mask(np.array([[1.0, np.nan], [2.0, 3.0]]), border=1)
```

5.4.2 Function `classify_policy_cells`

Classify each cell into a policy class based on slope/VRM/BPI.

Syntax

```
from mc_geomap_utils.macroregions import classify_policy_cells
classify_policy_cells(features: dict[str, np.ndarray],
                      slope_flat_deg: float = 5.0,
                      slope_steep_deg: float = 15.0,
                      vrm_steep: float = 0.06,
                      bpi_sigma: float | None = None,
                      bpi_sigma_mult: float = 1.0,
                      ridge_min_slope_deg: float = 3.0, # NEW
                      vrm_min_slope_deg: float = 7.0 # NEW
                      ) -> tuple[np.ndarray, float]
```

Inputs

```
features (dict[str, np.ndarray]): feature rasters including ``slope``, ``vrm``, and ``bpi
    ``.
slope_flat_deg (float): upper slope limit for flat class.
slope_steep_deg (float): lower slope limit for steep class.
vrm_steep (float): VRM threshold for steep classification.
bpi_sigma (float | None): override for BPI sigma (otherwise estimated).
bpi_sigma_mult (float): multiplier for BPI sigma thresholding.
ridge_min_slope_deg (float): minimum slope for ridge/valley class.
vrm_min_slope_deg (float): minimum slope for VRM-driven steep class.
```

Outputs

```
tuple[np.ndarray, float]: ``policy_map`` and computed ``sigma_bpi``.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import classify_policy_cells
>>> feats = {"slope": np.zeros((5, 5)), "vrm": np.zeros((5, 5)), "bpi": np.zeros((5, 5))}
>>> policy, sigma_bpi = classify_policy_cells(feats)
```

5.4.3 Function `grad_mag_deg`

Compute gradient magnitude of the slope surface in degrees.

Syntax

```
from mc_geomap_utils.macroregions import grad_mag_deg
grad_mag_deg(slope_deg: np.ndarray) -> np.ndarray
```

Inputs

```
slope_deg (np.ndarray): slope raster expressed in degrees.
```

Outputs

```
np.ndarray: gradient magnitude at each cell.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import grad_mag_deg
>>> grad = grad_mag_deg(np.zeros((3, 3)))
```

5.4.4 Function `edge_list_with_boundaries`

Collect boundary pixel coordinates for each touching superpixel pair.

Syntax

```
from mc_geomap_utils.macroregions import edge_list_with_boundaries
edge_list_with_boundaries(labels: np.ndarray) -> dict[tuple[int,int], list[tuple[int,int
    ]]]
```

Inputs

```
labels (np.ndarray): label raster describing superpixels.
```

Outputs

```
dict[tuple[int,int], list[tuple[int,int]]]: map from region pairs to seam coordinates.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import edge_list_with_boundaries
>>> edges = edge_list_with_boundaries(np.array([[1, 1], [2, 2]]))
```

5.4.5 Function `superpixel_stats`

Summarise each superpixel with area, class, and mean features.

Syntax

```
from mc_geomap_utils.macroregions import superpixel_stats
superpixel_stats(labels: np.ndarray, policy_map: np.ndarray,
                 feats: dict[str, np.ndarray]) -> dict[int, dict]
```

Inputs

```
labels (np.ndarray): superpixel label raster.
policy_map (np.ndarray): class map aligned to the labels.
feats (dict[str, np.ndarray]): feature rasters used for statistics.
```

Outputs

```
dict[int, dict]: mapping from region id to collected attributes.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import superpixel_stats
>>> labels = np.array([[1, 1], [2, 2]])
>>> policy = np.array([[1, 1], [2, 2]])
>>> feats = {"slope": np.zeros((2, 2)), "vrm": np.zeros((2, 2)), "bpi": np.zeros((2, 2))}
>>> stats = superpixel_stats(labels, policy, feats)
```

5.4.6 Function `rag_merge_slic`

Merge adjacent superpixels when classes agree and the boundary is weak.

Boundary strength gamma_ij is the mean |nabla(slope)| along the interface. Thresholds are class-wise percentiles; if a class has no edges, the global percentile is used.

Syntax

```
from mc_geomap_utils.macroregions import rag_merge_slic
rag_merge_slic(labels: np.ndarray,
               policy_map: np.ndarray,
               feats: dict[str, np.ndarray],
               area_cap_px: int = 40000,
               tau_pct: float = 98.0,
               min_area_px: int = 0) -> tuple[np.ndarray, dict[int, int]]
```

Inputs

```
labels (np.ndarray): SLIC superpixel label raster.
policy_map (np.ndarray): per-cell policy classes aligned to ``labels``.
feats (dict[str, np.ndarray]): feature rasters (expects ``"slope"``).
area_cap_px (int): maximum area for merged regions (pixels).
tau_pct (float): percentile used for boundary strength thresholds.
min_area_px (int): minimum area for macroregions (smaller ones are absorbed).
```

Outputs

```
tuple[np.ndarray, dict[int, int]]: macro label raster and mapping from macro id to class.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import rag_merge_slic
>>> labels = np.array([[1, 1], [2, 2]])
>>> policy = np.array([[1, 1], [2, 2]])
>>> feats = {"slope": np.zeros((2, 2))}
>>> macro, reg_cls = rag_merge_slic(labels, policy, feats)
```

5.4.7 Function `axial_mean_heading`

Compute the dominant heading within a region using slope/aspect weights.

Syntax

```
from mc_geomap_utils.macroregions import axial_mean_heading
axial_mean_heading(mask: np.ndarray, aspect: np.ndarray, slope: np.ndarray, along: bool)
    -> float
```

Inputs

```
mask (np.ndarray): boolean mask describing the macroregion.
aspect (np.ndarray): aspect raster measured in radians.
slope (np.ndarray): slope raster measured in radians.
along (bool): whether to align headings with the slope direction.
```

Outputs

```
float: heading angle expressed in radians.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import axial_mean_heading
>>> mask = np.ones((2, 2), dtype=bool)
>>> aspect = np.zeros((2, 2))
>>> slope = np.zeros((2, 2))
>>> axial_mean_heading(mask, aspect, slope, along=False)
0.0
```

5.4.8 Function `policy_for_class`

Derive default survey policy parameters for a macroregion class.

Syntax

```
from mc_geomap_utils.macroregions import policy_for_class
policy_for_class(cls: int, W: float, h0: float, hmin: float, betamax_deg: float,
    mean_slope_deg: float)
```

Inputs

```
cls (int): policy class identifier.
W (float): nominal swath width.
h0 (float): baseline altitude parameter.
hmin (float): minimum altitude permitted.
betamax_deg (float): upper bound for pitch/heading offsets.
mean_slope_deg (float): mean slope in degrees for the region.
```

Outputs

```
dict: dictionary containing spacing, altitude, and orientation flags.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.macroregions import policy_for_class
>>> policy_for_class(1, W=3.0, h0=5.0, hmin=2.0, betamax_deg=20.0, mean_slope_deg=4.0)
```

5.4.9 Function `policy_superpixel_consensus`

Within each SLIC superpixel, if one class occupies >=min_frac, assign the whole superpixel to it.

Syntax

```
from mc_geomap_utils.macroregions import policy_superpixel_consensus
policy_superpixel_consensus(policy: np.ndarray, labels: np.ndarray, min_frac: float =
    0.60) -> np.ndarray
```

Inputs

```
policy (np.ndarray): per-cell policy class raster.
labels (np.ndarray): superpixel labels aligned to ``policy``.
min_frac (float): minimum fraction for consensus assignment.
```

Outputs

```
np.ndarray: updated policy raster with superpixel consensus applied.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import policy_superpixel_consensus
>>> policy = np.array([[1, 1], [0, 2]])
>>> labels = np.array([[1, 1], [1, 2]])
>>> out = policy_superpixel_consensus(policy, labels, min_frac=0.5)
```

5.4.10 Function `policy_mode_filter`

Mode filter on nonzero classes to remove salt-and-pepper noise.

Syntax

```
from mc_geomap_utils.macroregions import policy_mode_filter
policy_mode_filter(policy: np.ndarray, size: int = 3) -> np.ndarray
```

Inputs

```
policy (np.ndarray): policy class raster.
size (int): neighborhood size (square window).
```

Outputs

```
np.ndarray: filtered policy raster.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import policy_mode_filter
>>> out = policy_mode_filter(np.array([[1, 0], [2, 2]]), size=3)
```

5.4.11 Function `summarize_regions`

Compile region-wise metrics and recommended survey policies.

Syntax

```
from mc_geomap_utils.macroregions import summarize_regions
summarize_regions(macro: np.ndarray,
                  reg_cls: dict[int,int],
                  feats: dict[str,np.ndarray],
                  W: float,
                  h0: float,
                  hmin: float,
                  betamax_deg: float,
                  cell_m: float) -> list[dict]
```

Inputs

```
macro (np.ndarray): macroregion label raster.
reg_cls (dict[int, int]): mapping from macro id to policy class.
feats (dict[str, np.ndarray]): feature rasters used for stats.
W (float): nominal swath width.
h0 (float): baseline altitude parameter.
hmin (float): minimum altitude permitted.
betamax_deg (float): maximum allowed heading offset.
cell_m (float): raster cell size in metres.
```

Outputs

```
list[dict]: per-region policy summary rows.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import summarize_regions
>>> macro = np.array([[1, 1], [2, 2]])
>>> reg_cls = {1: 1, 2: 2}
>>> feats = {"slope": np.zeros((2, 2)), "aspect": np.zeros((2, 2)), "vrm": np.zeros((2,
    2)), "bpi": np.zeros((2, 2))}
>>> rows = summarize_regions(macro, reg_cls, feats, W=3.0, h0=5.0, hmin=2.0, betamax_deg
    =20.0, cell_m=1.0)
```

5.4.12 Function `export_regions_csv`

Write macroregion summaries to CSV.

Syntax

```
from mc_geomap_utils.macroregions import export_regions_csv
export_regions_csv(rows: list[dict], path) -> None
```

Inputs

```
rows (list[dict]): list of per-region policy records.
path (str | Path): destination CSV path.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.macroregions import export_regions_csv
>>> export_regions_csv([{"id": 1, "cls": 1}], "regions.csv")
```

5.4.13 Function `absorb_small_islands`

Replace very small regions with their most common neighboring label.

Syntax

```
from mc_geomap_utils.macroregions import absorb_small_islands
absorb_small_islands(labels: np.ndarray, min_area_px: int) -> np.ndarray
```

Inputs

```
labels (np.ndarray): macroregion label raster.
min_area_px (int): regions with area < min_area_px are absorbed.
```

Outputs

```
np.ndarray: relabeled raster.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import absorb_small_islands
>>> labels = np.array([[1, 1], [1, 2]])
>>> absorb_small_islands(labels, min_area_px=2)
```

5.4.14 Function `smooth_macro_labels`

Morphologically smooth jagged macro boundaries (closing+opening per region).

Regions are processed in descending area so larger regions keep priority when overlaps occur.

Syntax

```
from mc_geomap_utils.macroregions import smooth_macro_labels
smooth_macro_labels(labels: np.ndarray, radius_px: int = 2) -> np.ndarray
```

Inputs

```
labels (np.ndarray): macroregion label raster.
radius_px (int): structuring element radius in pixels.
```

Outputs

```
np.ndarray: smoothed macroregion labels.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.macroregions import smooth_macro_labels
>>> smooth_macro_labels(np.array([[1, 1], [1, 2]]), radius_px=1)
```

5.4.15 Class `UF`

Lightweight union-find structure for merging region adjacency graph components.

Method `find` Locate the canonical parent for node x with path compression.

Syntax

```
find(self, x)
```

Inputs

```
x (int): node identifier to locate.
```

Outputs

```
int: current root representative for x.
```

Raises Not specified.

Example

```
>>> uf = UF([1, 2])
>>> uf.find(1)
1
```

Method `union` Merge the sets containing nodes a and b.

Syntax

```
union(self, a, b)
```

Inputs

```
a (int): first node to merge.
b (int): second node to merge.
```

Outputs

```
int: identifier of the resulting root.
```

Raises Not specified.

Example

```
>>> uf = UF([1, 2])
>>> uf.union(1, 2)
1
```

5.5 Module `mc_geomap_utils.manipulation`

No description provided.

5.5.1 Function `shift_point_cloud`

Translate a point cloud so a chosen origin maps to (0, 0, ...).

Syntax

```
from mc_geomap_utils.manipulation import shift_point_cloud
shift_point_cloud(points: NDArray[np.floating], *, mode: Mode | str = Mode.FIRST, origin:
     Iterable[float] | None = None) -> NDArray[np.float64]
```

Inputs

```
points (NDArray[np.floating]): input coordinates with shape ``(N, >=2)``.
mode (Mode | str): origin-selection strategy (first/min/custom point).
origin (Iterable[float] | None): explicit origin coordinates when ``mode`` is custom.
```

Outputs

```
NDArray[np.float64]: shifted copy of the input point cloud.
```

Raises

```
ValueError: If the input shape is invalid or the mode/origin are incompatible.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.manipulation import shift_point_cloud
>>> pts = np.array([[1.0, 2.0], [3.0, 4.0]])
>>> shift_point_cloud(pts, mode="min")
```

5.5.2 Function `grid_to_point_cloud`

Convert a 2-D raster into "(x, y, z)" point samples with optional reprojection.

Syntax

```
from mc_geomap_utils.manipulation import grid_to_point_cloud
grid_to_point_cloud(
    raster: np.ndarray,
    transform: Any,
    *,
    src_crs: Any | None = None,
    dst_crs: Any | None = None,
) -> NDArray[np.float64]
```

Inputs

```
raster (np.ndarray): grid of Z values (shape ``rows x cols``).
transform (Any): affine transform describing pixel center coordinates.
src_crs (Any | None): CRS describing the raster coordinates (defaults to ``EPSG:4326``).
dst_crs (Any | None): CRS for the output XY values (skip reprojection when ``None``).
```

Outputs

```
NDArray[np.float64]: flattened point cloud with columns ``[x, y, z]``.
```

Raises

```
ValueError: If the input raster is invalid or contains no finite samples.
RuntimeError: If reprojection is requested but ``pyproj`` is unavailable.
TypeError: If the transform object is missing required affine attributes.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.manipulation import grid_to_point_cloud
>>> raster = np.array([[1.0, 2.0], [3.0, 4.0]])
>>> class T: a=1; b=0; c=0; d=0; e=-1; f=0
>>> pts = grid_to_point_cloud(raster, T())
```

5.5.3 Class `Mode`

Strategies for choosing the translation origin.

No public methods were found.

5.6 Module `mc_geomap_utils.planning`

No description provided.

5.6.1 Function `region_mask`

Return a boolean mask for a single macroregion id.

Syntax

```
from mc_geomap_utils.planning import region_mask
region_mask(macro_labels: np.ndarray, rid: int) -> np.ndarray
```

Inputs

```
macro_labels (np.ndarray): macroregion label raster.
rid (int): region id to extract.
```

Outputs

```
np.ndarray: boolean mask where ``macro_labels == rid``.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.planning import region_mask
>>> region_mask(np.array([[1, 2], [2, 2]]), 2)
```

5.6.2 Function `generate_serpentine_tracks_polygon`

Generate parallel strips by intersecting an AOI polygon with evenly spaced lines.

Syntax

```
from mc_geomap_utils.planning import generate_serpentine_tracks_polygon
generate_serpentine_tracks_polygon(mask: np.ndarray,
                                   transform: Any,
                                   psi_deg: float,
                                   spacing_m: float,
                                   *,
                                   min_seg_len_m: float | None = None) -> List[List[
                                       Tuple[float, float]]]
```

Inputs

```
mask (np.ndarray): boolean AOI mask.
transform (Any): affine transform mapping pixel centers to world coords.
psi_deg (float): heading angle in degrees.
spacing_m (float): spacing between tracks in metres.
min_seg_len_m (float | None): minimum segment length to keep.
```

Outputs

```
List[List[Tuple[float, float]]]: list of polylines, each a list of ``(x, y)`` points.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.planning import generate_serpentine_tracks_polygon
>>> mask = np.ones((5, 5), dtype=bool)
>>> tracks = generate_serpentine_tracks_polygon(mask, transform=None, psi_deg=0.0,
    spacing_m=1.0)
```

5.6.3 Function `generate_serpentine_tracks`

Raster-native generation of serpentine strips inside a binary mask.

Syntax

```
from mc_geomap_utils.planning import generate_serpentine_tracks
generate_serpentine_tracks(mask: np.ndarray,
                           transform: Any,
                           psi_deg: float,
                           spacing_m: float,
                           cell_m: float,
                           min_seg_len_m: float | None = None,
                           band_tol_frac: float = 0.35) -> List[List[Tuple[float,float
                               ]]]
```

Inputs

```
mask (np.ndarray): boolean AOI mask.
transform (Any): affine transform mapping pixel centers to world coords.
psi_deg (float): heading angle in degrees.
spacing_m (float): spacing between tracks in metres.
cell_m (float): raster cell size in metres.
```

```
min_seg_len_m (float | None): minimum segment length to keep.
band_tol_frac (float): band tolerance as a fraction of spacing.
```

Outputs

```
List[List[Tuple[float, float]]]: list of polylines in serpentine order.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.planning import generate_serpentine_tracks
>>> mask = np.ones((5, 5), dtype=bool)
>>> tracks = generate_serpentine_tracks(mask, transform=None, psi_deg=0.0, spacing_m=1.0,
     cell_m=1.0)
```

5.6.4 Function `attach_altitude_and_tilt`

Sample DEM under each waypoint and attach z and tilt.

Syntax

```
from mc_geomap_utils.planning import attach_altitude_and_tilt
attach_altitude_and_tilt(polylines: List[List[Tuple[float,float]]],
                         dem: np.ndarray,
                         transform: Any,
                         h_m: float,
                         beta_deg: float) -> List[List[Dict]]
```

Inputs

```
polylines (List[List[Tuple[float, float]]]): list of track polylines.
dem (np.ndarray): DEM raster for altitude sampling.
transform (Any): affine transform mapping pixel centers to world coords.
h_m (float): altitude offset to add above the DEM.
beta_deg (float): tilt angle (degrees) assigned to each waypoint.
```

Outputs

```
List[List[Dict]]: list of enriched waypoints with ``x``, ``y``, ``z``, and ``tilt_deg``.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.planning import attach_altitude_and_tilt
>>> polylines = [[(0.0, 0.0), (1.0, 1.0)]]
>>> dem = np.zeros((5, 5))
>>> out = attach_altitude_and_tilt(polylines, dem, transform=None, h_m=2.0, beta_deg=5.0)
```

5.6.5 Function `estimate_turn_count`

Approximate number of turns (ends of segments) for quick reporting.

Syntax

```
from mc_geomap_utils.planning import estimate_turn_count
estimate_turn_count(tracks: List[List[Tuple[float,float]]]) -> int
```

Inputs

```
tracks (List[List[Tuple[float, float]]]): list of track polylines.
```

Outputs

```
int: estimated number of turns (segments minus one).
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import estimate_turn_count
>>> estimate_turn_count([[(0, 0), (1, 0)], [(1, 0), (2, 0)]])
1
```

5.6.6 Function `polyline_length_m`

Compute total length of a polyline.

Syntax

```
from mc_geomap_utils.planning import polyline_length_m
polyline_length_m(points_xy: List[Tuple[float,float]]) -> float
```

Inputs

```
points_xy (List[Tuple[float, float]]): list of ``(x, y)`` points.
```

Outputs

```
float: polyline length in metres.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import polyline_length_m
>>> polyline_length_m([(0, 0), (3, 4)])
5.0
```

5.6.7 Function `tracks_total_length_m`

Compute total length across multiple polylines.

Syntax

```
from mc_geomap_utils.planning import tracks_total_length_m
tracks_total_length_m(tracks: List[List[Tuple[float,float]]]) -> float
```

Inputs

```
tracks (List[List[Tuple[float, float]]]): list of polylines.
```

Outputs

```
float: total length in metres.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import tracks_total_length_m
>>> tracks_total_length_m([[(0, 0), (1, 0)], [(1, 0), (1, 1)]])
2.0
```

5.6.8 Function `merge_tracks_continuous`

Concatenate ordered segments into a single continuous path (bridging jumps).

Each input list is assumed to be ordered along-strip. We connect the end of one segment to the start of the next to keep one continuous polyline per region.

Syntax

```
from mc_geomap_utils.planning import merge_tracks_continuous
merge_tracks_continuous(tracks: List[List[Tuple[float, float]]]) -> List[Tuple[float,
    float]]
```

Inputs

```
tracks (List[List[Tuple[float, float]]]): ordered track segments.
```

Outputs

```
List[Tuple[float, float]]: merged continuous polyline.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import merge_tracks_continuous
>>> merged = merge_tracks_continuous([[(0, 0), (1, 0)], [(2, 0), (3, 0)]])
```

5.6.9 Function `straighten_tracks`

Replace each strip polyline with a straight line between its endpoints.

Syntax

```
from mc_geomap_utils.planning import straighten_tracks
straighten_tracks(tracks: List[List[Tuple[float, float]]]) -> List[List[Tuple[float,
    float]]]
```

Inputs

```
tracks (List[List[Tuple[float, float]]]): list of polylines.
```

Outputs

```
List[List[Tuple[float, float]]]: list of straightened polylines.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import straighten_tracks
>>> out = straighten_tracks([[(0, 0), (1, 1), (2, 0)]])
```

5.6.10 Function `smooth_polyline`

Apply a simple moving average to a polyline to soften stair-steps.

Syntax

```
from mc_geomap_utils.planning import smooth_polyline
smooth_polyline(points: List[Tuple[float, float]], window: int = 5) -> List[Tuple[float,
    float]]
```

Inputs

```
points (List[Tuple[float, float]]): polyline points.
window (int): moving average window size (odd, >=3).
```

Outputs

```
List[Tuple[float, float]]: smoothed polyline points.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import smooth_polyline
>>> out = smooth_polyline([(0, 0), (1, 1), (2, 0)], window=3)
```

5.6.11 Function `simplify_polyline_rdp`

Douglas-Peucker simplification to reduce vertices while keeping shape.

Syntax

```
from mc_geomap_utils.planning import simplify_polyline_rdp
simplify_polyline_rdp(points: List[Tuple[float, float]], epsilon: float) -> List[Tuple[
    float, float]]
```

Inputs

```
points (List[Tuple[float, float]]): polyline points.
epsilon (float): maximum deviation allowed (metres).
```

Outputs

```
List[Tuple[float, float]]: simplified polyline points.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.planning import simplify_polyline_rdp
>>> out = simplify_polyline_rdp([(0, 0), (1, 0.1), (2, 0)], epsilon=0.2)
```

5.7 Module `mc_geomap_utils.plotting`

No description provided.

5.7.1 Function `show_plot`

Render one or multiple XY series using Matplotlib subplots.

Syntax

```
from mc_geomap_utils.plotting import show_plot
show_plot(data: Any, shared_axes: bool = True, separate_axes: bool = True) -> Figure
```

Inputs

```
data (Any): sequence, mapping, or alternating list describing XY pairs.
shared_axes (bool): whether generated subplots share their axes.
separate_axes (bool): split series across subplots (True) or overlay on one axis (False).
```

Outputs

```
Figure: Matplotlib figure containing the requested plot layout.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.plotting import show_plot
>>> fig = show_plot({"x": [0, 1], "y": [1, 2]})
```

5.7.2 Function `save_mosaic`

Save a 2x2 mosaic from up to 4 image paths.

Syntax

```
from mc_geomap_utils.plotting import save_mosaic
save_mosaic(paths: list[Path], out_path: Path, titles: list[str] | None = None, figsize
    =(12, 10)) -> None
```

Inputs

```
paths (list[Path]): image file paths to include (up to 4).
out_path (Path): output image path for the mosaic.
titles (list[str] | None): optional per-image titles.
figsize: Matplotlib figure size.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from pathlib import Path
>>> from mc_geomap_utils.plotting import save_mosaic
>>> save_mosaic([Path("a.png"), Path("b.png")], Path("mosaic.png"))
```

5.7.3 Function `save_mosaic_1x2`

Save a 1x2 mosaic from up to 2 image paths, padding sizes to match.

Syntax

```
from mc_geomap_utils.plotting import save_mosaic_1x2
save_mosaic_1x2(paths: list[Path], out_path: Path, titles: list[str] | None = None,
    figsize=(12, 5)) -> None
```

Inputs

```
paths (list[Path]): image file paths to include (up to 2).
out_path (Path): output image path for the mosaic.
titles (list[str] | None): optional per-image titles.
figsize: Matplotlib figure size.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from pathlib import Path
>>> from mc_geomap_utils.plotting import save_mosaic_1x2
>>> save_mosaic_1x2([Path("a.png"), Path("b.png")], Path("mosaic_1x2.png"))
```

5.7.4 Function `plot_point_cloud`

Send a point cloud to Open3D for interactive inspection.

Syntax

```
from mc_geomap_utils.plotting import plot_point_cloud
plot_point_cloud(points: np.ndarray, *, draw_fn: Callable[[list[Any]], None] | None =
    None) -> None
```

Inputs

```
points (np.ndarray): XYZ coordinates to render.
draw_fn (Callable[[list[Any]], None] | None): optional custom Open3D draw function.
```

Outputs

```
None: the side effect is an Open3D window displaying the cloud.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_point_cloud
>>> plot_point_cloud(np.random.rand(100, 3))
```

5.7.5 Function `plot_mesh`

Plot a triangulated surface using Matplotlib's 3D toolkit.

Syntax

```
from mc_geomap_utils.plotting import plot_mesh
plot_mesh(points: np.ndarray, tri: Delaunay, *, show: bool = True) -> Figure
```

Inputs

```
points (np.ndarray): vertex coordinates with columns x, y, z.
tri (Delaunay): triangulation object describing the mesh faces.
show (bool): display the plot interactively after creation.
```

Outputs

```
Figure: Matplotlib figure containing the rendered mesh.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_mesh
>>> from scipy.spatial import Delaunay
>>> pts = np.random.rand(10, 3)
>>> tri = Delaunay(pts[:, :2])
>>> fig = plot_mesh(pts, tri, show=False)
```

5.7.6 Function `plot_dem`

Display or export a DEM using masked shading.

Syntax

```
from mc_geomap_utils.plotting import plot_dem
plot_dem(
    dem: np.ndarray,
    transform: Any | None = None,
    title: str = "DEM␣(Z)",
    cmap: str = "terrain",
    save: Path | None = None,
    show: bool = False,
    colorbar: bool = True,
)
```

Inputs

```
dem (np.ndarray): digital elevation model raster.
transform (Any | None): affine transform for deriving plot extent.
title (str): title text applied to the figure.
cmap (str): Matplotlib colormap name used to render elevation.
save (Path | None): optional location for saving the PNG output.
show (bool): display the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_dem
>>> plot_dem(np.random.rand(10, 10), show=False)
```

5.7.7 Function `plot_feature_map`

Render a derived feature raster (slope, VRM, BPI, etc.).

Syntax

```
from mc_geomap_utils.plotting import plot_feature_map
plot_feature_map(arr: np.ndarray, name: str, transform: Any | None = None, save: Path |
    None = None, show: bool = False)
```

Inputs

```
arr (np.ndarray): feature raster to visualise.
name (str): human-readable feature name (controls scaling and label).
transform (Any | None): affine transform for computing the extent.
save (Path | None): optional output path for a saved image.
show (bool): whether to display the plot interactively.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_feature_map
>>> plot_feature_map(np.random.rand(10, 10), "slope", show=False)
```

5.7.8 Function `plot_labels`

Overlay colorized labels on an optional base raster and export/display.

Syntax

```
from mc_geomap_utils.plotting import plot_labels
plot_labels(labels: np.ndarray, transform: Any | None = None, base: np.ndarray | None =
    None,
            base_name: str = "Base", save: Path | None = None, show: bool = False,
            alpha_labels: float = 0.55, draw_boundaries: bool = True)
```

Inputs

```
labels (np.ndarray): integer label image to visualize.
transform (Any | None): spatial transform for extent computation.
base (np.ndarray | None): optional background raster for context.
base_name (str): descriptive name of the background raster.
save (Path | None): location for saving the PNG overlay.
show (bool): show the figure interactively when True.
alpha_labels (float): opacity applied to the label overlay.
draw_boundaries (bool): draw boundary outlines when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_labels
>>> plot_labels(np.random.randint(0, 3, (10, 10)), show=False)
```

5.7.9 Function `plot_policy_map`

Render a policy-class raster with a categorical legend.

Syntax

```
from mc_geomap_utils.plotting import plot_policy_map
plot_policy_map(policy_map: np.ndarray, transform, *, title="Policy␣classes",
                  save=None, show=False)
```

Inputs

```
policy_map (np.ndarray): integer policy class raster.
transform: affine transform for computing plot extent.
title (str): figure title.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_policy_map
>>> plot_policy_map(np.random.randint(0, 5, (10, 10)), transform=None, show=False)
```

5.7.10 Function `plot_macro_overlay`

Overlay macroregion boundaries on a base raster.

Syntax

```
from mc_geomap_utils.plotting import plot_macro_overlay
plot_macro_overlay(base: np.ndarray, macro_labels: np.ndarray, transform, *, title="
    Macroregions␣over␣base", save=None, show=False)
```

Inputs

```
base (np.ndarray): base raster (e.g., DEM) for context.
macro_labels (np.ndarray): macroregion labels aligned to ``base``.
transform: affine transform for computing plot extent.
title (str): figure title.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_macro_overlay
>>> plot_macro_overlay(np.random.rand(10, 10), np.ones((10, 10), dtype=int), transform=
    None, show=False)
```

5.7.11 Function `plot_macro_with_seeds`

Render DEM + macro boundaries + seed markers.

Syntax

```
from mc_geomap_utils.plotting import plot_macro_with_seeds
plot_macro_with_seeds(
    dem: np.ndarray,
    macro_labels: np.ndarray,
    transform,
    seeds_rc: list[tuple[int, int]] | None = None,
    *,
    macro_transform=None,
    seeds_debug: dict | None = None,
    title: str = "DEM␣+␣Seeds␣+␣Macro␣overlay",
    save=None,
    show=False,
) -> None
```

Inputs

```
dem (np.ndarray): DEM raster.
macro_labels (np.ndarray): macroregion labels.
transform: affine transform for DEM extent.
seeds_rc (list[tuple[int, int]] | None): optional seed pixel coords.
macro_transform: optional transform for macro labels (defaults to ``transform``).
seeds_debug (dict | None): optional debug seed sets.
title (str): figure title.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_macro_with_seeds
>>> plot_macro_with_seeds(np.random.rand(5, 5), np.ones((5, 5), dtype=int), transform=
    None, show=False)
```

5.7.12 Function `plot_region_headings`

Draw one heading arrow per macroregion, overlaid on a base raster.

Syntax

```
from mc_geomap_utils.plotting import plot_region_headings
plot_region_headings(
    macro_labels: np.ndarray,
    psi_deg_per_id: dict[int, float],
    transform,
    *,
    base: np.ndarray, # e.g., np.degrees(feats["slope"])
    title: str = "Region␣headings",
```

```
    save=None,
    show: bool = False,
    arrow_scale: float = 8.0, # quiver scale (bigger -> shorter arrows)
) -> None
```

Inputs

```
macro_labels (np.ndarray): macroregion label raster (0 = NoData).
psi_deg_per_id (dict[int, float]): mapping from region id to heading in degrees.
transform: affine transform for computing extents (None for pixel coords).
base (np.ndarray): background raster (e.g., slope in degrees).
title (str): figure title.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
arrow_scale (float): quiver scale (larger -> shorter arrows).
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_region_headings
>>> plot_region_headings(np.ones((5, 5), dtype=int), {1: 45.0}, transform=None, base=np.
    zeros((5, 5)), show=False)
```

5.7.13 Function `plot_dem_with_policy_overlay`

Overlay a policy-class map on top of a DEM.

Syntax

```
from mc_geomap_utils.plotting import plot_dem_with_policy_overlay
plot_dem_with_policy_overlay(
    dem: np.ndarray,
    policy_map: np.ndarray,
    transform,
    *,
    title: str = "DEM␣+␣Policy␣overlay",
    base_cmap: str = "terrain",
    hillshade: bool = False, # set True if you like shaded relief
    policy_alpha: float = 0.45, # transparency of the overlay
    use_rgba_overlay: bool = True, # robust path that builds RGBA image explicitly
    save=None,
    show=False
)
```

Inputs

```
dem (np.ndarray): DEM raster.
policy_map (np.ndarray): policy class raster.
transform: affine transform for computing extents.
title (str): figure title.
base_cmap (str): colormap for the DEM base layer.
hillshade (bool): whether to use hillshade for the base layer.
policy_alpha (float): transparency for policy overlay.
use_rgba_overlay (bool): whether to build an explicit RGBA overlay.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_dem_with_policy_overlay
>>> plot_dem_with_policy_overlay(np.random.rand(10, 10), np.zeros((10, 10), dtype=int),
    transform=None, show=False)
```

5.7.14 Function `make_quicklook_figures`

Generate quicklook PNGs for DEM, point density, features, and labels.

Syntax

```
from mc_geomap_utils.plotting import make_quicklook_figures
make_quicklook_figures(
    dem: np.ndarray,
    count: np.ndarray,
    features: dict[str, np.ndarray] | None = None,
    labels: np.ndarray | None = None,
    transform: Any | None = None,
    outdir: Path | str = "quicklooks",
    cell: float | None = None,
    show: bool = False
) -> dict[str, Path]
```

Inputs

```
dem (np.ndarray): elevation raster used as the base layer.
count (np.ndarray): per-cell point count raster for density plots.
features (dict[str, np.ndarray] | None): optional feature rasters such as slope, VRM, or
    BPI.
labels (np.ndarray | None): label raster to overlay on top of a base layer.
transform (Any | None): affine transform for computing spatial extents.
outdir (Path | str): output directory where PNGs are written.
cell (float | None): grid cell size used for labelling/metadata (currently informational)
    .
show (bool): display the generated Matplotlib figures interactively.
```

Outputs

```
dict[str, Path]: mapping between quicklook identifier and saved PNG path.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import make_quicklook_figures
>>> dem = np.random.rand(5, 5)
>>> count = np.ones((5, 5))
>>> outputs = make_quicklook_figures(dem, count, outdir="quicklooks", show=False)
```

5.7.15 Function `plot_tracks_overlay`

Overlay survey tracks on a DEM.

Syntax

```
from mc_geomap_utils.plotting import plot_tracks_overlay
plot_tracks_overlay(dem: np.ndarray,
                    transform,
                    tracks: list[list[tuple[float,float]]],
                    *,
                    title: str = "Planned␣tracks",
                    color: str = "k",
                    linewidth: float = 1.0,
                    save=None,
                    show=False)
```

Inputs

```
dem (np.ndarray): DEM raster.
transform: affine transform for computing extents.
tracks (list[list[tuple[float, float]]]): list of track polylines.
title (str): figure title.
color (str): track color.
linewidth (float): track line width.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_tracks_overlay
>>> plot_tracks_overlay(np.random.rand(5, 5), transform=None, tracks=[[(0, 0), (1, 1)]],
    show=False)
```

5.7.16 Function `plot_tracks_overlay_by_region`

Plot serpentine tracks with a distinct color per region (one continuous line each).

Syntax

```
from mc_geomap_utils.plotting import plot_tracks_overlay_by_region
plot_tracks_overlay_by_region(
    dem: np.ndarray,
    transform,
    tracks_by_region: dict[int, list[list[tuple[float, float]]]],
    *,
    title: str = "Planned␣tracks",
    linewidth: float = 1.0,
    save=None,
    show=False,
) -> None
```

Inputs

```
dem (np.ndarray): DEM raster.
transform: affine transform for computing extents.
tracks_by_region (dict[int, list[list[tuple[float, float]]]]): mapping from region id to
    tracks.
title (str): figure title.
linewidth (float): track line width.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_tracks_overlay_by_region
>>> tracks = {1: [[(0, 0), (1, 1)]]}
>>> plot_tracks_overlay_by_region(np.random.rand(5, 5), transform=None, tracks_by_region=
    tracks, show=False)
```

5.7.17 Function `plot_macro_tracks_overlay`

Plot DEM with macroregion boundaries and region-colored tracks.

Syntax

```
from mc_geomap_utils.plotting import plot_macro_tracks_overlay
plot_macro_tracks_overlay(
    dem: np.ndarray,
    macro_labels: np.ndarray,
    transform,
    tracks_by_region: dict[int, list[list[tuple[float, float]]]],
    *,
```

```
    title: str = "Macroregions␣+␣Tracks",
    linewidth: float = 1.0,
    save=None,
    show=False,
) -> None
```

Inputs

```
dem (np.ndarray): DEM raster.
macro_labels (np.ndarray): macroregion labels aligned to ``dem``.
transform: affine transform for computing extents.
tracks_by_region (dict[int, list[list[tuple[float, float]]]]): mapping from region id to
    tracks.
title (str): figure title.
linewidth (float): track line width.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_macro_tracks_overlay
>>> tracks = {1: [[(0, 0), (1, 1)]]}
>>> plot_macro_tracks_overlay(np.random.rand(5, 5), np.ones((5, 5), dtype=int), transform
    =None, tracks_by_region=tracks, show=False)
```

5.7.18 Function `plot_dem_with_polygon`

Plot DEM with an outline of the polygon AOI and optional tracks.

Syntax

```
from mc_geomap_utils.plotting import plot_dem_with_polygon
plot_dem_with_polygon(
    dem: np.ndarray,
    transform,
    polygon_xy: np.ndarray,
    *,
    tracks: dict[int, list[list[tuple[float, float]]]] | None = None,
    macro_labels: np.ndarray | None = None,
    macro_transform=None,
    title: str = "DEM␣+␣AOI",
    save=None,
    show=False,
) -> None
```

Inputs

```
dem (np.ndarray): DEM raster.
transform: affine transform for computing extents.
polygon_xy (np.ndarray): polygon vertices in world coordinates.
tracks (dict[int, list[list[tuple[float, float]]]] | None): optional tracks by region.
macro_labels (np.ndarray | None): optional macroregion labels.
macro_transform: optional transform for macro labels.
title (str): figure title.
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_dem_with_polygon
>>> plot_dem_with_polygon(np.random.rand(5, 5), transform=None, polygon_xy=np.array
    ([[0,0],[1,0],[1,1],[0,1]]), show=False)
```

5.7.19 Function `plot_primitives_overlay`

Draw detected primitives on top of the DEM.

Rows may contain either the new fields (“prim_cx_px“, “prim_cy_px“, “prim_r_m“) or legacy ones (“cx_px“, “cy_px“, “r_m“). New names are preferred.

Syntax

```
from mc_geomap_utils.plotting import plot_primitives_overlay
plot_primitives_overlay(
    dem: np.ndarray,
    transform,
    rows: list[dict],
    macro_labels: np.ndarray,
    *,
    title: str = "DEM␣+␣Primitives␣overlay",
    base_cmap: str = "terrain",
    save=None,
    show=False,
    macro_transform=None,
)
```

Inputs

```
dem (np.ndarray): DEM raster.
transform: affine transform for computing extents.
rows (list[dict]): region rows containing primitive metadata.
macro_labels (np.ndarray): macroregion labels aligned to ‘‘dem‘‘.
title (str): figure title.
base_cmap (str): colormap for the DEM base layer.
```

```
save: optional output path for saving the figure.
show (bool): show the figure interactively when True.
macro_transform: optional transform for macro labels.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_primitives_overlay
>>> plot_primitives_overlay(np.random.rand(5, 5), transform=None, rows=[], macro_labels=
    np.zeros((5, 5), dtype=int), show=False)
```

5.7.20 Function `plot_knoll_diagnostics`

Diagnostic plot for DEM-based knoll detection.

Shows DEM background, macroregion boundaries, and seed points at different filtering stages (when available).

Syntax

```
from mc_geomap_utils.plotting import plot_knoll_diagnostics
plot_knoll_diagnostics(
    dem: np.ndarray,
    transform: Any,
    feats: dict[str, np.ndarray],
    macro_labels: np.ndarray | None,
    *,
    macro_transform=None,
    save: Path | str | None = None,
    show: bool = False,
) -> Figure
```

Inputs

```
dem (np.ndarray): DEM raster.
transform (Any): affine transform for computing extents.
feats (dict[str, np.ndarray]): feature rasters used by knoll detection.
macro_labels (np.ndarray | None): macroregion labels aligned to ``dem``.
macro_transform: optional transform for macro labels.
save (Path | str | None): optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
Figure: Matplotlib figure containing the diagnostic plot.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_knoll_diagnostics
>>> feats = {"slope": np.zeros((5, 5))}
>>> fig = plot_knoll_diagnostics(np.random.rand(5, 5), transform=None, feats=feats,
    macro_labels=None, show=False)
```

5.7.21 Function `plot_primitives_3d`

Render DEM as a surface with fitted primitive footprints (ellipse wires).

Syntax

```
from mc_geomap_utils.plotting import plot_primitives_3d
plot_primitives_3d(
    dem: np.ndarray,
    transform: Any,
    rows: list[dict],
    *,
    title: str = "DEM␣+␣Primitive␣fit␣(3D)",
    save: Path | str | None = None,
    show: bool = False,
) -> Figure | None
```

Inputs

```
dem (np.ndarray): DEM raster.
transform (Any): affine transform for computing world coordinates.
rows (list[dict]): region rows containing primitive ellipse metadata.
title (str): figure title.
save (Path | str | None): optional output path for saving the figure.
show (bool): show the figure interactively when True.
```

Outputs

```
Figure | None: Matplotlib figure, or ``None`` if ``transform`` is missing.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_primitives_3d
>>> fig = plot_primitives_3d(np.random.rand(5, 5), transform=None, rows=[], show=False)
```

5.7.22 Function `plot_primitives_3d_interactive`

Export an interactive HTML (Plotly) with DEM surface and fitted primitives.

Syntax

```
from mc_geomap_utils.plotting import plot_primitives_3d_interactive
plot_primitives_3d_interactive(
    dem: np.ndarray,
    transform: Any,
    rows: list[dict],
    *,
    title: str = "DEM␣+␣Primitive␣fit␣(3D,␣interactive)",
    save: Path | str | None = None,
) -> Any
```

Inputs

```
dem (np.ndarray): DEM raster.
transform (Any): affine transform for computing world coordinates.
rows (list[dict]): region rows containing primitive metadata.
title (str): figure title.
save (Path | str | None): optional output path for saving the HTML.
```

Outputs

```
Any: Plotly figure (or ``None`` if dependencies are unavailable).
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_primitives_3d_interactive
>>> fig = plot_primitives_3d_interactive(np.random.rand(5, 5), transform=None, rows=[],
    save=None)
```

5.7.23 Function `plot_tracks_3d_interactive`

Export an interactive 3D HTML with DEM surface and colored tracks.

Syntax

```
from mc_geomap_utils.plotting import plot_tracks_3d_interactive
plot_tracks_3d_interactive(
    dem: np.ndarray,
    transform: Any,
    tracks_by_region: dict[int, list[list[tuple[float, float]]]],
    *,
    title: str = "DEM␣+␣Tracks␣(3D,␣interactive)",
    save: Path | str | None = None,
    fancy: bool = False,
) -> Any
```

Inputs

```
dem (np.ndarray): DEM raster.
transform (Any): affine transform for computing world coordinates.
tracks_by_region (dict[int, list[list[tuple[float, float]]]]): mapping from region id to
    tracks.
title (str): figure title.
save (Path | str | None): optional output path for saving the HTML.
fancy (bool): enable enhanced styling.
```

Outputs

```
Any: Plotly figure (or ``None`` if dependencies are unavailable).
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_tracks_3d_interactive
>>> fig = plot_tracks_3d_interactive(np.random.rand(5, 5), transform=None,
    tracks_by_region={}, save=None)
```

5.7.24 Function `plot_macro_labels_3d_interactive`

Export an interactive 3D HTML with DEM surface colored by macroregion IDs or policy classes.

Syntax

```
from mc_geomap_utils.plotting import plot_macro_labels_3d_interactive
plot_macro_labels_3d_interactive(
    dem: np.ndarray,
    macro_labels: np.ndarray,
    transform: Any,
    *,
    title: str = "DEM␣+␣Macroregions␣(3D,␣interactive)",
    save: Path | str | None = None,
    policy_map: np.ndarray | None = None,
    fancy: bool = False,
) -> Any
```

Inputs

```
dem (np.ndarray): DEM raster.
macro_labels (np.ndarray): macroregion labels aligned to ``dem``.
transform (Any): affine transform for computing world coordinates.
title (str): figure title.
save (Path | str | None): optional output path for saving the HTML.
policy_map (np.ndarray | None): optional policy classes for coloring.
fancy (bool): enable enhanced styling.
```

Outputs

```
Any: Plotly figure (or ``None`` if dependencies are unavailable).
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_macro_labels_3d_interactive
>>> fig = plot_macro_labels_3d_interactive(np.random.rand(5, 5), np.ones((5, 5), dtype=
    int), transform=None, save=None)
```

5.7.25 Function `plot_knoll_ellipses_3d`

Interactive 3D DEM with cylindrical ellipses for knoll footprints.

Syntax

```
from mc_geomap_utils.plotting import plot_knoll_ellipses_3d
plot_knoll_ellipses_3d(
    dem: np.ndarray,
    transform: Any,
    rows: list[dict],
    *,
    title: str = "DEM␣+␣Knoll␣ellipses␣(3D)",
    save=None,
    alpha_surface: float = 0.8,
)
```

Inputs

```
dem (np.ndarray): DEM raster.
transform (Any): affine transform for computing world coordinates.
rows (list[dict]): region rows containing ellipse metadata.
title (str): figure title.
save: optional output path for saving the HTML.
alpha_surface (float): surface transparency.
```

Outputs

```
Any: Plotly figure.
```

Raises

```
ImportError: If Plotly is unavailable.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.plotting import plot_knoll_ellipses_3d
>>> fig = plot_knoll_ellipses_3d(np.random.rand(5, 5), transform=None, rows=[], save=None
    )
```

5.7.26 Class `PlotDataSource`

Container helping turn assorted inputs into labelled XY series.

Method `from_input` Normalise many possible user inputs into a consistent mapping.

Syntax

```
from_input(cls, raw: Any) -> "PlotDataSource"
```

Inputs

```
raw (Any): mapping of labels to arrays or alternating key/value sequence.
```

Outputs

```
PlotDataSource: wrapper holding the parsed series.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.plotting import PlotDataSource
>>> ds = PlotDataSource.from_input({"x": [0, 1], "y": [1, 2]})
```

Method `iter_xy_pairs` Yield paired X/Y arrays for plotting convenience.

Syntax

```
iter_xy_pairs(self) -> list[tuple[str, np.ndarray, np.ndarray]]
```

Inputs

```
None
```

Outputs

```
list[tuple[str, np.ndarray, np.ndarray]]: tuples containing label, x-array, and y-array.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.plotting import PlotDataSource
>>> ds = PlotDataSource.from_input({"x": [0, 1], "y": [1, 2]})
>>> ds.iter_xy_pairs()
```

5.8 Module `mc_geomap_utils.pointcloud`

No description provided.

5.8.1 Function `delaunay_mesh`

Build a 2.5D Delaunay triangulation from XY coordinates.

Syntax

```
from mc_geomap_utils.pointcloud import delaunay_mesh
delaunay_mesh(points: np.ndarray) -> Delaunay
```

Inputs

```
points (np.ndarray): input point cloud with columns ``x, y, z``.
```

Outputs

```
Delaunay: triangulation object describing XY simplices.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import delaunay_mesh
>>> tri = delaunay_mesh(np.random.rand(10, 3))
```

5.8.2 Function `mesh_to_obj`

Write a triangulated mesh to disk in OBJ format.

Syntax

```
from mc_geomap_utils.pointcloud import mesh_to_obj
mesh_to_obj(points: np.ndarray, tri: Delaunay, filename: Path | str = "mesh.obj") -> Path
```

Inputs

```
points (np.ndarray): vertex positions used by the triangulation.
tri (Delaunay): triangulation describing mesh faces.
filename (Path | str): destination file path for the OBJ export.
```

Outputs

```
Path: path to the created OBJ file.
```

Raises

```
RuntimeError: If ``trimesh`` is unavailable.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import delaunay_mesh, mesh_to_obj
>>> pts = np.random.rand(10, 3)
>>> tri = delaunay_mesh(pts)
>>> _ = mesh_to_obj(pts, tri, "mesh.obj")
```

5.8.3 Function `delaunay_to_open3d_mesh`

Convert a SciPy triangulation to an Open3D TriangleMesh instance.

Syntax

```
from mc_geomap_utils.pointcloud import delaunay_to_open3d_mesh
delaunay_to_open3d_mesh(points: np.ndarray, tri: Delaunay) -> Any
```

Inputs

```
points (np.ndarray): vertex coordinates (N x 3).
tri (Delaunay): triangulation whose simplices form mesh faces.
```

Outputs

```
Any: Open3D ``TriangleMesh`` ready for visualization.
```

Raises

```
RuntimeError: If ``open3d`` is unavailable.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import delaunay_mesh, delaunay_to_open3d_mesh
>>> pts = np.random.rand(10, 3)
>>> tri = delaunay_mesh(pts)
>>> mesh = delaunay_to_open3d_mesh(pts, tri)
```

5.8.4 Function `grid_to_dem`

Rasterize an irregular point cloud into a DEM and count grid.

Syntax

```
from mc_geomap_utils.pointcloud import grid_to_dem
grid_to_dem(points: np.ndarray, cell: float = 0.5, min_count: int = 3, agg: str = "median
    ")
```

Inputs

```
points (np.ndarray): input XYZ coordinates.
cell (float): grid resolution in the XY plane.
min_count (int): minimum samples required to keep a raster cell.
agg (str): aggregation strategy ("median", "mean", or percentile ``pXX``).
```

Outputs

```
tuple[np.ndarray, np.ndarray, Any]: DEM array, per-cell counts, and raster transform.
```

Raises

```
ValueError: If an unsupported aggregation strategy is requested.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import grid_to_dem
>>> pts = np.array([[0.0, 0.0, 1.0], [0.1, 0.1, 2.0]])
>>> dem, counts, transform = grid_to_dem(pts, cell=0.5)
```

5.8.5 Function `dem_to_features`

Derive terrain features (slope, curvature, VRM, BPI) from a DEM.

Syntax

```
from mc_geomap_utils.pointcloud import dem_to_features
dem_to_features(dem: np.ndarray, cell: float = 0.5, smooth_sigma_cells: float = 1.0,
                  vrm_win_cells: int = 11)
```

Inputs

```
dem (np.ndarray): raster DEM with NaNs marking gaps.
cell (float): grid spacing in metres.
smooth_sigma_cells (float): Gaussian sigma (in cells) for derivative smoothing.
```

Outputs

```
dict[str, np.ndarray]: mapping of feature names to float32 rasters.
```

Raises

```
ValueError: If the DEM contains no finite samples.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import dem_to_features
>>> dem = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=float)
>>> feats = dem_to_features(dem, cell=1.0)
```

5.8.6 Function `feature_to_slic_segmentation`

Segment feature rasters with SLIC to produce superpixel labels.

Syntax

```
from mc_geomap_utils.pointcloud import feature_to_slic_segmentation
feature_to_slic_segmentation(features: dict, cell: float, W: float = 3.0, compactness:
    float = 30.0)
```

Inputs

```
features (dict): dictionary containing feature rasters (slope, k1, k2, vrm, bpi).
cell (float): grid spacing used when estimating segment sizes.
W (float): nominal swath width controlling the desired segment area.
compactness (float): SLIC compactness parameter balancing color and spatial distance.
```

Outputs

```
np.ndarray: integer label image (same shape as the DEM features).
```

Raises

```
RuntimeError: If scikit-image is unavailable.
ValueError: If no valid pixels exist in the input features.
```

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.pointcloud import feature_to_slic_segmentation
>>> feats = {"slope": np.ones((10, 10)), "k1": np.zeros((10, 10)), "k2": np.zeros((10,
    10)), "vrm": np.zeros((10, 10)), "bpi": np.zeros((10, 10))}
>>> labels = feature_to_slic_segmentation(feats, cell=1.0)
```

5.9 Module `mc_geomap_utils.primitives`

No description provided.

5.9.1 Function `detect_knolls`

Detect knoll-like blobs on |nablaslope| using thresholding and shape filters.

Syntax

```
from mc_geomap_utils.primitives import detect_knolls
detect_knolls(
    feats: dict[str, np.ndarray],
    macro_labels: np.ndarray,
    rows: list[dict],
    *,
    cell_m: float,
    W_m: float,
    slope_ring_deg: float = 12.0,
    circ_cv_max: float = 0.20,
    rmin_mult: float = 1.5,
    rmax_mult: float = 4.0,
    spiral_rmin_m: float = 1.0,
    size_relax: float = 1.0,
    use_grad_seeds: bool = True, # retained for CLI compatibility
    grad_seed_pct: float = 85.0,
    dem: np.ndarray | None = None,
) -> List[Knoll]
```

Inputs

```
feats (dict[str, np.ndarray]): feature rasters (expects ``"slope"`` and optionally ``"dem
    "``).
macro_labels (np.ndarray): macroregion label raster aligned to ``feats``.
rows (list[dict]): region metadata rows (used for default spacing/heading).
cell_m (float): raster cell size in metres.
W_m (float): nominal swath width in metres.
slope_ring_deg (float): slope ring threshold in degrees.
circ_cv_max (float): maximum allowed circularity coefficient of variation.
rmin_mult (float): minimum radius multiplier relative to ``W_m``.
rmax_mult (float): maximum radius multiplier relative to ``W_m``.
spiral_rmin_m (float): minimum spiral radius for forced footprints.
size_relax (float): softening factor for size limits.
use_grad_seeds (bool): when True, seed from gradient magnitude peaks.
grad_seed_pct (float): percentile for gradient thresholding when seeding.
dem (np.ndarray | None): optional DEM for height metadata.
```

Outputs

```
List[Knoll]: detected knoll objects sorted by score.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.primitives import detect_knolls
>>> feats = {"slope": np.zeros((10, 10))}
>>> knolls = detect_knolls(feats, np.zeros((10, 10), dtype=int), [], cell_m=1.0, W_m=3.0,
     spiral_rmin_m=1.0)
```

5.9.2 Function `detect_knolls_plane_descent`

Detect knolls by descending a plane and tracking connected super-level sets.

Syntax

```
from mc_geomap_utils.primitives import detect_knolls_plane_descent
detect_knolls_plane_descent(
    dem: np.ndarray,
    macro_labels: np.ndarray,
    rows: list[dict],
    *,
    cell_m: float,
    W_m: float,
    rmin_mult: float = 1.5,
    rmax_mult: float = 4.0,
    prominence_min_m: float = 3.0,
    descent_dmax_m: float = 30.0,
    dz_m: float = 0.5,
    size_relax: float = 1.0,
    cls_by_id: Dict[int, int] | None = None,
    steep_only: bool = False,
) -> List[Knoll]
```

Inputs

```
dem (np.ndarray): DEM raster used for plane descent.
macro_labels (np.ndarray): macroregion label raster aligned to ``dem``.
rows (list[dict]): region metadata rows (used for default spacing/heading).
cell_m (float): raster cell size in metres.
W_m (float): nominal swath width in metres.
rmin_mult (float): minimum radius multiplier relative to ``W_m``.
rmax_mult (float): maximum radius multiplier relative to ``W_m``.
prominence_min_m (float): minimum prominence for a seed peak.
descent_dmax_m (float): maximum descent depth.
dz_m (float): vertical step for plane descent.
size_relax (float): softening factor for size limits.
cls_by_id (Dict[int, int] | None): optional macroregion class lookup.
steep_only (bool): when True, keep only steep class regions.
```

Outputs

```
List[Knoll]: detected knoll objects sorted by score.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.primitives import detect_knolls_plane_descent
>>> dem = np.random.rand(10, 10)
>>> knolls = detect_knolls_plane_descent(dem, np.zeros((10, 10), dtype=int), [], cell_m
    =1.0, W_m=3.0)
```

5.9.3 Function `augment_regions_with_primitives`

Attach primitive metadata (spiral around knoll) to region rows.

Syntax

```
from mc_geomap_utils.primitives import augment_regions_with_primitives
augment_regions_with_primitives(
    rows: list[dict],
    macro_labels: np.ndarray,
    feats: dict[str, np.ndarray],
    transform: Any,
    *,
    cell_m: float,
    W_m: float,
    slope_ring_deg: float,
    circ_cv_max: float,
    rmin_mult: float,
    rmax_mult: float,
    size_relax: float = 1.0,
    spiral_rmin_m: float,
    use_grad_seeds: bool = True,
    grad_seed_pct: float = 85.0,
    dem: np.ndarray | None = None,
    method: str = "ring",
    prominence_min_m: float = 3.0,
    descent_dmax_m: float = 30.0,
    dz_m: float = 0.5,
    steep_only: bool = False,
    force_seeds_xy: list[tuple[float, float]] | None = None,
    force_plane_z: float | None = None,
    force_plane_z_list: list[float] | None = None,
    force_min_diam_m: float = 0.0,
) -> list[dict]
```

Inputs

```
rows (list[dict]): region records to augment in place.
macro_labels (np.ndarray): macroregion label raster.
feats (dict[str, np.ndarray]): feature rasters (expects ``"slope"`` and optionally ``"dem
    "``).
```

```
transform (Any): affine transform mapping pixel centers to world coords.
cell_m (float): raster cell size in metres.
W_m (float): nominal swath width in metres.
slope_ring_deg (float): slope ring threshold in degrees.
circ_cv_max (float): maximum allowed circularity coefficient of variation.
rmin_mult (float): minimum radius multiplier relative to ``W_m``.
rmax_mult (float): maximum radius multiplier relative to ``W_m``.
size_relax (float): softening factor for size limits.
spiral_rmin_m (float): minimum spiral radius for forced footprints.
use_grad_seeds (bool): whether to seed from gradient magnitude peaks.
grad_seed_pct (float): percentile for gradient thresholding when seeding.
dem (np.ndarray | None): optional DEM for height metadata.
method (str): detection method (``"ring"`` or ``"plane"``).
prominence_min_m (float): minimum prominence for plane descent.
descent_dmax_m (float): maximum descent depth.
dz_m (float): vertical step for plane descent.
steep_only (bool): when True, keep only steep class regions.
force_seeds_xy (list[tuple[float, float]] | None): force-add seeds (world coords).
force_plane_z (float | None): optional target plane for forced footprints.
force_plane_z_list (list[float] | None): optional per-seed plane heights.
force_min_diam_m (float): minimum diameter for forced footprints.
```

Outputs

```
list[dict]: updated region rows (same list, augmented in place).
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.primitives import augment_regions_with_primitives
>>> rows = [{"id": 1}]
>>> feats = {"slope": np.zeros((5, 5))}
>>> out = augment_regions_with_primitives(rows, np.ones((5, 5), dtype=int), feats,
    transform=None, cell_m=1.0, W_m=3.0,
... slope_ring_deg=12.0, circ_cv_max=0.2, rmin_mult=1.5, rmax_mult=4.0, spiral_rmin_m
    =1.0)
```

5.9.4 Class `Knoll`

No description provided.

No public methods were found.

5.10 Module `mc_geomap_utils.spiral`

No description provided.

5.10.1 Function `archimedean_spiral`

Generate a constant-spacing Archimedean spiral polyline.

The spiral follows “r(theta) = r_out - b*theta“ with “2pib = d_m“ and proceeds inward from “r_out“ to “r_in“.

Syntax

```
from mc_geomap_utils.spiral import archimedean_spiral
archimedean_spiral(center_xy: Tuple[float,float],
                      r_out: float,
                      d_m: float,
                      r_in: float,
                      *,
                      step_m: float | None = None) -> List[Tuple[float,float]]
```

Inputs

```
center_xy (Tuple[float, float]): spiral center in world coordinates.
r_out (float): outer radius in metres.
d_m (float): spacing between successive turns in metres.
r_in (float): inner radius cutoff in metres.
step_m (float | None): approximate step length along the curve (defaults to ``0.5 * d_m
    ``).
```

Outputs

```
List[Tuple[float, float]]: list of ``(x, y)`` points defining the spiral polyline.
```

Raises Not specified.

Example

```
>>> from mc_geomap_utils.spiral import archimedean_spiral
>>> pts = archimedean_spiral((0.0, 0.0), r_out=10.0, d_m=2.0, r_in=1.0)
```

5.10.2 Function `clip_polyline_to_mask`

Clip a polyline to a raster mask and return contiguous valid segments.

Syntax

```
from mc_geomap_utils.spiral import clip_polyline_to_mask
clip_polyline_to_mask(points_xy: List[Tuple[float,float]],
                         mask: np.ndarray,
                         transform: Any) -> List[List[Tuple[float,float]]]
```

Inputs

```
points_xy (List[Tuple[float, float]]): input polyline points in world coordinates.
mask (np.ndarray): boolean raster mask (True = keep).
transform (Any): affine transform mapping pixel centers to world coordinates.
```

Outputs

```
List[List[Tuple[float, float]]]: list of polyline segments contained within the mask.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_geomap_utils.spiral import clip_polyline_to_mask
>>> mask = np.ones((10, 10), dtype=bool)
>>> class T: a=1; b=0; c=0; d=0; e=-1; f=0
>>> segs = clip_polyline_to_mask([(0.5, -0.5), (5.5, -5.5)], mask, T())
```

6 Package `mc_io_utils`

6.1 Module `mc_io_utils`

IO utilities.

Exports: - scan_channels - list_channel_fields - from_imu_kearfott_compas - list_files - load_csv_timeseries_to_mc_kinematics - PointFileMode - McPointFileMode - clear_console - setup_logging_from_yaml - check_output_file - sum_total_bytes - estimate_total_bytes

LCM/adapters exports are available when optional dependencies are installed.

No public functions or classes were found in this module.

6.2 Module `mc_io_utils.adapters`

LCM channel adapters.

Functions: - from_imu_kearfott_compas(imu)

6.2.1 Function `from_imu_kearfott_compas`

Build KinematicsData from an IMU_KEARFOTT_COMPAS dict.

Populates: - body.pos.psi/theta/phi from quaternion orientation (rad) - body.vel.x/y/z from angRate (rad/s) - body.acc.x/y/z from xyz_ddot (m/s^2) - t_s from lcm_timestamp (or header timestamp) - t_datetime from header timestamp when available

Syntax

```
from mc_io_utils.adapters import from_imu_kearfott_compas
from_imu_kearfott_compas(imu: Dict) -> McKinematicsData
```

Inputs

```
imu (Dict): decoded LCM message dict with fields like ``orientation``, ``angRate``,
``xyz_ddot``, and timestamps.
```

Outputs

```
McKinematicsData: populated kinematics data with body frame fields.
```

Raises

```
ValueError: If required fields are missing.
```

Example

```
>>> from mc_io_utils.adapters import from_imu_kearfott_compas
>>> data = from_imu_kearfott_compas(imu_dict)
```

6.3 Module `mc_io_utils.io`

IO helpers for point clouds and raster outputs.

Functions: - list_files(folder, pattern="*", recursive=False) - load_points(file_path, mode=...) - load_csv_timeseries_to_mc_kinematics(file_path, columns, ...) - save_points(file_path, points) - load_grd(...) - write_geotiff(...)

6.3.1 Function `load_csv_timeseries_to_mc_kinematics`

Load a CSV timeseries and map columns into “McKinematicsData“.

Syntax

```
from mc_io_utils.io import load_csv_timeseries_to_mc_kinematics
load_csv_timeseries_to_mc_kinematics(
    file_path: Path | str,
    columns: Sequence[str],
    *,
    delimiter: str = ",",
    has_header: bool = False,
    skip_rows: int = 0,
    datetime_format: str | None = None,
    units: Optional[Dict[str, str]] = None,
)
```

Inputs

```
file_path (Path | str): CSV file path.
columns (Sequence[str]): ordered mapping for CSV columns (left-to-right), e.g.
    ``["t_s", "body.pos.phi", "body.pos.theta", "body.pos.psi"]``.
    Supported aliases for ignored columns: ``""``, ``"_"``, ``"-"``, ``"ignore"``.
    Supported kinematics keys: ``<frame>.<group>.<field>`` with
    ``frame in {ned, body}``, ``group in {pos, vel, acc}``, and
    ``field in {x, y, z, psi, theta, phi}``.
    A 2-part key like ``pos.x`` defaults to frame ``body``.
delimiter (str): CSV delimiter (default: ``,``,).
has_header (bool): whether to skip one header row.
skip_rows (int): additional rows to skip after optional header.
datetime_format (str | None): ``strptime`` format for ``t_datetime`` values.
    If omitted, ISO-8601 parsing is used.
units (Optional[Dict[str, str]]): optional unit overrides merged into default units.
```

Outputs

```
McKinematicsData: populated kinematics structure.
```

Raises

```
ValueError: If column mapping is invalid or CSV rows are malformed.
KeyError: If neither ``t_s`` nor ``t_datetime`` is provided.
```

Example

```
>>> from mc_io_utils.io import load_csv_timeseries_to_mc_kinematics
>>> data = load_csv_timeseries_to_mc_kinematics(
... "imu.csv",
... columns=["t_s", "body.pos.phi", "body.pos.theta", "body.pos.psi"],
... )
```

6.3.2 Function `load_points`

Load a whitespace-delimited point cloud into a NumPy array.

Syntax

```
from mc_io_utils.io import load_points
load_points(file_path: Path, *, mode: PointFileMode | str = PointFileMode.VERTICAL) ->
    NDArray[np.float64]
```

Inputs

```
file_path (Path): absolute path to the source XYZ text file.
mode (PointFileMode | str): "vertical" for one point per line, "horizontal" for per-axis
    rows.
```

Outputs

```
NDArray[np.float64]: array with columns ``x, y, z`` describing each point.
```

Raises

```
ValueError: If the file contents are malformed or incompatible with the chosen layout.
FileNotFoundError: If ``file_path`` does not exist.
```

Example

```
>>> from pathlib import Path
>>> from mc_io_utils.io import load_points
>>> pts = load_points(Path("points.xyz"))
```

6.3.3 Function `list_files`

List files in a folder matching a glob pattern.

Syntax

```
from mc_io_utils.io import list_files
list_files(folder: str | Path, pattern: str = "*", recursive: bool = False) -> List[Path]
```

Inputs

```
folder (str | Path): directory to search.
pattern (str): glob pattern, e.g. "*.txt".
recursive (bool): when True, search recursively (uses rglob).
```

Outputs

```
List[Path]: matching file paths.
```

Raises

```
FileNotFoundError: If ``folder`` does not exist or is not a directory.
```

Example

```
>>> from mc_io_utils.io import list_files
>>> files = list_files("data", "*.txt", recursive=True)
```

6.3.4 Function `save_points`

Persist a point cloud next to the original file with a "*_cleaned" suffix.

Syntax

```
from mc_io_utils.io import save_points
save_points(file_path: Path, points: np.ndarray) -> Path
```

Inputs

```
file_path (Path): original input file whose directory will host the cleaned copy.
points (np.ndarray): coordinates to be written (shape N x 3).
```

Outputs

```
Path: filesystem path of the generated ``*_cleaned.txt`` file.
```

Raises Not specified.

Example

```
>>> from pathlib import Path
>>> import numpy as np
>>> from mc_io_utils.io import save_points
>>> out_path = save_points(Path("points.xyz"), np.zeros((3, 3)))
```

6.3.5 Function `load_grd`

Load a GMT/NetCDF ".grd" file and return its raster plus metadata.

Syntax

```
from mc_io_utils.io import load_grd
load_grd(
    file_path: Path | str,
    *,
    assume_wgs84: bool = True,
) -> tuple[np.ndarray, Any, Any | None]
```

Inputs

```
file_path (Path | str): path to the grid file on disk.
assume_wgs84 (bool): inject ``EPSG:4326`` when the dataset lacks a CRS.
```

Outputs

```
tuple[np.ndarray, Any, Any | None]: data array, affine transform, and CRS (if present).
```

Raises

```
FileNotFoundError: If the grid file does not exist.
RuntimeError: If ``rasterio`` is unavailable.
```

Example

```
>>> from mc_io_utils.io import load_grd
>>> data, transform, crs = load_grd("surface.grd")
```

6.3.6 Function `write_geotiff`

Persist a NumPy array as a single-band GeoTIFF (with .npy fallback).

Syntax

```
from mc_io_utils.io import write_geotiff
write_geotiff(
    path: Path | str,
    arr: np.ndarray,
    transform: Any,
    *,
    crs: str | None = None,
    nodata: float | None = np.nan,
) -> None
```

Inputs

```
path (Path | str): destination file path for the raster.
arr (np.ndarray): image data to be written.
transform (Any): affine transform describing pixel-to-world mapping.
crs (str | None): coordinate reference system identifier.
nodata (float | None): value representing no-data cells.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_io_utils.io import write_geotiff
>>> write_geotiff("out.tif", np.zeros((10, 10)), transform=None)
```

6.3.7 Class `PointFileMode`

Layout of coordinates inside a plain-text point cloud file.

No public methods were found.

6.4 Module `mc_io_utils.lcm`

LCM log IO utilities.

Functions: - scan_channels(log_dir, file_pattern, max_events=None) - scan_logs_detailed(log_paths, ...) - list_channel_fields(log_path, channels, lcm_packages=None) - list_channel_fields_in_dir(log_dir, file_pattern, channel, lcm_packages=None) - lcmlog_to_pickle(log_dir, file_pattern="*.00", channels=None, out_path="lcmlogs_merged.pkl", lcm_packages=None, format="pickle") - main() # CLI entrypoint

6.4.1 Function `scan_channels`

Scan raw LCM log files and return channel counts.

Syntax

```
from mc_io_utils.lcm import scan_channels
scan_channels(log_dir: str, file_pattern: str, max_events: int | None = None) -> Dict[str
    , int]
```

Inputs

```
log_dir (str): directory containing LCM log files.
file_pattern (str): glob pattern to match log files (e.g. ``"*.00"``).
max_events (int | None): optional cap on events to scan across files.
```

Outputs

```
Dict[str, int]: mapping of channel name to message count.
```

Raises

```
FileNotFoundError: If the directory or matching files are not found.
RuntimeError: If the LCM Python bindings are unavailable.
```

Example

```
>>> from mc_io_utils.lcm import scan_channels
>>> counts = scan_channels("lcmlogs", "*.00", max_events=1000)
```

6.4.2 Function `scan_logs_detailed`

Scan LCM logs and return counts, timestamp quality, and optional sync metrics.

Syntax

```
from mc_io_utils.lcm import scan_logs_detailed
scan_logs_detailed(
    log_paths: Iterable[str | Path],
    *,
    max_events_per_file: int | None = None,
    gap_warning_seconds: float = 1.0,
    sonar_channels: Optional[Iterable[str]] = None,
    gps_channels: Optional[Iterable[str]] = None,
    rph_channels: Optional[Iterable[str]] = None,
) -> LCMScanStats
```

Inputs

```
log_paths: iterable of LCM log file paths to scan in order.
max_events_per_file: optional cap of scanned events per file.
gap_warning_seconds: threshold used for ``gap_over_threshold`` counters.
sonar_channels: channels used as sync reference events.
gps_channels: channels treated as GPS time references.
rph_channels: channels treated as RPH/attitude time references.
```

Outputs

```
LCMScanStats: aggregate and per-file metrics.
```

Raises

```
RuntimeError: If LCM bindings are unavailable.
ValueError: If ``log_paths`` is empty.
FileNotFoundError: If any provided file does not exist.
```

Example

```
>>> from mc_io_utils.lcm import scan_logs_detailed
>>> stats = scan_logs_detailed(["log.00"], gap_warning_seconds=0.5)
>>> stats.total_events >= 0
True
```

6.4.3 Function `list_channel_fields`

Decode one message per channel and list available fields.

Returns a dict: {channel: {"lcmtype": str|None, "fields": [..], "header_fields": [..]}}

Syntax

```
from mc_io_utils.lcm import list_channel_fields
list_channel_fields(
    log_path: str,
    channels: Iterable[str],
    lcm_packages: Optional[list[str]] = None,
) -> Dict[str, Dict[str, object]]
```

Inputs

```
log_path (str): path to a single LCM log file.
channels (Iterable[str]): channel names to inspect.
lcm_packages (Optional[list[str]]): optional list of Python LCM packages to import.
```

Outputs

```
Dict[str, Dict[str, object]]: per-channel metadata including lcmtype and field lists.
```

Raises

```
RuntimeError: If the LCM Python bindings are unavailable.
```

Example

```
>>> from mc_io_utils.lcm import list_channel_fields
>>> info = list_channel_fields("log.00", ["IMU_KEARFOTT_COMPAS"])
```

6.4.4 Function `list_channel_fields_in_dir`

Convenience wrapper to read one log file in a directory and list channel fields.

Syntax

```
from mc_io_utils.lcm import list_channel_fields_in_dir
list_channel_fields_in_dir(
    log_dir: str,
    file_pattern: str,
    channel: str,
    lcm_packages: Optional[list[str]] = None,
) -> Dict[str, object]
```

Inputs

```
log_dir (str): directory containing LCM logs.
file_pattern (str): glob pattern used to select a log file.
channel (str): channel name to inspect.
lcm_packages (Optional[list[str]]): optional list of Python LCM packages to import.
```

Outputs

```
Dict[str, object]: dictionary with ``channel``, ``lcmtype``, and ``fields`` entries.
```

Raises

```
FileNotFoundError: If the directory or matching files are not found.
```

Example

```
>>> from mc_io_utils.lcm import list_channel_fields_in_dir
>>> entry = list_channel_fields_in_dir("lcmlogs", "*.00", "IMU_KEARFOTT_COMPAS")
```

6.4.5 Function `lcmlog_to_pickle`

Convert raw LCM logs to a merged output file.

Syntax

```
from mc_io_utils.lcm import lcmlog_to_pickle
lcmlog_to_pickle(
    log_dir: str,
    file_pattern: str = "*.00",
    channels: Optional[Iterable[str]] = None,
    out_path: str = "lcmlogs_merged.pkl",
    lcm_packages: Optional[list[str]] = None,
    format: str = "pickle",
    *,
    verbose: bool = True,
) -> Path
```

Inputs

```
log_dir: directory containing LCM log files.
file_pattern: glob pattern for log files (default: *.00).
channels: optional iterable of channel names to include. If None, includes all.
out_path: output path.
lcm_packages: optional list of LCM packages to load.
format: output format (currently supports "pickle" or "pkl").
verbose: pass-through verbosity flag to the parser.
```

Outputs

```
Path: path to the written output file.
```

Raises

```
RuntimeError: If ``navlib`` cannot be imported.
ValueError: If ``format`` is unsupported.
```

Example

```
>>> from mc_io_utils.lcm import lcmlog_to_pickle
>>> out = lcmlog_to_pickle("lcmlogs", file_pattern="*.00", out_path="merged.pkl")
>>> out.name
'merged.pkl'
```

6.4.6 Function `main`

CLI entrypoint for LCM log utilities.

Syntax

```
from mc_io_utils.lcm import main
main() -> int
```

Inputs

```
None
```

Outputs

```
int: exit code (0 for success).
```

Raises Not specified.

Example

```
>>> from mc_io_utils.lcm import main
>>> isinstance(main(), int)
True
```

6.4.7 Class `ChannelTimingStats`

Timestamp quality metrics for one LCM channel.

Method `add_event` Update channel timing metrics with one event timestamp.

Syntax

```
add_event(self, timestamp_us: int, gap_warning_us: int) -> None
```

Inputs

```
timestamp_us (int): event timestamp in microseconds.
gap_warning_us (int): threshold above which a gap is flagged.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> stats = ChannelTimingStats()
>>> stats.add_event(1_000_000, gap_warning_us=500_000)
```

6.4.8 Class `SyncAgeStats`

Simple sonar-vs-navigation timing age statistics in microseconds.

Method `observe` Update sync-age counters for one sonar event.

Syntax

```
observe(self, sonar_timestamp_us: int, last_gps_timestamp_us: int, last_rph_timestamp_us:
     int) -> None
```

Inputs

```
sonar_timestamp_us (int): sonar event timestamp in microseconds.
last_gps_timestamp_us (int): most recent GPS timestamp in microseconds.
last_rph_timestamp_us (int): most recent RPH timestamp in microseconds.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> sync = SyncAgeStats()
>>> sync.observe(10_000_000, last_gps_timestamp_us=9_500_000, last_rph_timestamp_us=9
    _700_000)
```

6.4.9 Class `FileScanStats`

Per-file counters and timestamp range.

No public methods were found.

6.4.10 Class `LCMScanStats`

Aggregate scan output for one or more LCM logs.

No public methods were found.

6.5 Module `mc_io_utils.runtime`

Runtime utilities for console, logging, and LCM log progress estimation.

Functions: - clear_console() - setup_logging_from_yaml(cfg_path, logs_dir="logs", prefix="compas_run") - check_output_file(output_path, remove_existing=True) - sum_total_bytes(file_path) - estimate_total_bytes(file_path, sample_events=20000)

6.5.1 Function `clear_console`

Clear the current terminal screen.

The command used depends on the host OS: "cls" on Windows and "clear" on Unix-like systems.

Syntax

```
from mc_io_utils.runtime import clear_console
clear_console() -> None
```

Inputs

```
None
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from mc_io_utils.runtime import clear_console
>>> clear_console()
```

6.5.2 Function `setup_logging_from_yaml`

Configure Python logging from a YAML config file.

Any handler named "file" or "json_file" that contains a "filename" entry is rewritten to include a timestamped basename under "logs_dir".

Syntax

```
from mc_io_utils.runtime import setup_logging_from_yaml
setup_logging_from_yaml(
    cfg_path: str | PathLike[str],
    logs_dir: str | PathLike[str] = "logs",
    prefix: str = "compas_run",
) -> None
```

Inputs

```
cfg_path (str | PathLike[str]): path to a YAML logging configuration file.
logs_dir (str | PathLike[str]): directory where timestamped log files are created.
prefix (str): prefix used for generated log filenames.
```

Outputs

```
None
```

Raises

```
RuntimeError: If ``PyYAML`` is not available.
```

Example

```
>>> from mc_io_utils.runtime import setup_logging_from_yaml
>>> setup_logging_from_yaml("logging.yaml", logs_dir="logs", prefix="run")
```

6.5.3 Function `check_output_file`

Validate an output path by creating/truncating the file.

This helper ensures the parent directory exists, optionally removes an existing file, and verifies write permission with a short open/close cycle.

Syntax

```
from mc_io_utils.runtime import check_output_file
check_output_file(output_path: Path, *, remove_existing: bool = True) -> bool
```

Inputs

```
output_path (Path): destination path to validate.
remove_existing (bool): when ``True``, remove an existing file before validating.
```

Outputs

```
bool: ``True`` if the output file can be created/written, ``False`` otherwise.
```

Raises Not specified.

Example

```
>>> from pathlib import Path
>>> from mc_io_utils.runtime import check_output_file
>>> ok = check_output_file(Path("out/data.pkl"))
>>> isinstance(ok, bool)
True
```

6.5.4 Function `sum_total_bytes`

Return exact payload bytes by scanning every event in an LCM log.

Syntax

```
from mc_io_utils.runtime import sum_total_bytes
sum_total_bytes(file_path: str | Path) -> Optional[int]
```

Inputs

```
file_path (str | Path): path to an LCM log file.
```

Outputs

```
Optional[int]: exact sum of ``len(event.data)`` across all events, or
``None`` if scanning fails (for example missing LCM bindings).
```

Raises Not specified.

Example

```
>>> from mc_io_utils.runtime import sum_total_bytes
>>> total = sum_total_bytes("session.00")
```

6.5.5 Function `estimate_total_bytes`

Estimate total payload bytes from an initial sample of events.

The estimator reads up to "sample_events" events, computes average payload size, approximates event count from on-disk file size, and returns an estimated total payload size. Falls back to coarse file-size heuristics if parsing is unavailable.

Syntax

```
from mc_io_utils.runtime import estimate_total_bytes
estimate_total_bytes(file_path: str | Path, sample_events: int = 20000) -> Optional[int]
```

Inputs

```
file_path (str | Path): path to an LCM log file.
sample_events (int): maximum number of events used for sampling.
```

Outputs

```
Optional[int]: estimated payload bytes, or ``None`` when estimation fails.
```

Raises Not specified.

Example

```
>>> from mc_io_utils.runtime import estimate_total_bytes
>>> estimate_total_bytes("session.00", sample_events=5000)
```

7 Package mc_libs_index

7.1 Module mc_libs_index

Index of mc_* libraries and their functions.

Exports: - list_libraries() -> str (tree) - list_functions(library) -> str (tree) - list_examples(library) - list_all() -> str (tree) - list_all_auto(base_path=None) - discover_libraries(base_path=None) - get_help(func_path) - get_example(func_path) - tree(libraries=None) - print_tree(libraries=None) - list_cli_commands()

7.1.1 Function list_cli_commands

Return a tree-like list of CLI commands, usage, and targets.

Syntax

```
from mc_libs_index import list_cli_commands
list_cli_commands() -> str
```

Inputs

```
None
```

Outputs

```
str: tree-style list of CLI commands.
```

Raises Not specified.

Example

```
>>> from mc_libs_index import list_cli_commands
>>> print(list_cli_commands())
```

7.2 Module mc_libs_index.index

Index of mc_* libraries, functions, and examples.

7.2.1 Function discover_libraries

Discover mc_* libraries in a given path (defaults to project root).

Syntax

```
from mc_libs_index.index import discover_libraries
discover_libraries(base_path: str | Path | None = None) -> List[str]
```

Inputs

```
base_path (str | Path | None): base directory to scan (defaults to project root).
```

Outputs

```
List[str]: sorted list of discovered package names.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import discover_libraries
>>> libs = discover_libraries()
```

7.2.2 Function `build_registry`

Build a registry by introspecting mc_* libraries.

Syntax

```
from mc_libs_index.index import build_registry
build_registry(base_path: str | Path | None = None) -> Dict[str, List[dict]]
```

Inputs

```
base_path (str | Path | None): base directory to scan (defaults to project root).
```

Outputs

```
Dict[str, List[dict]]: mapping from library name to entries with ``name`` and ``example
    ``.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import build_registry
>>> registry = build_registry()
```

7.2.3 Function `list_libraries`

Return a tree-like string of discovered mc_* libraries.

Syntax

```
from mc_libs_index.index import list_libraries
list_libraries() -> str
```

Inputs

```
None
```

Outputs

```
str: tree-style list of library names.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import list_libraries
>>> print(list_libraries())
```

7.2.4 Function `list_functions`

Return a tree-like string of functions for a library.

Syntax

```
from mc_libs_index.index import list_functions
list_functions(library: str) -> str
```

Inputs

```
library (str): library name, e.g. ``"mc_data_utils"``.
```

Outputs

```
str: tree-style list of functions for the library.
```

Raises

```
KeyError: If the library is unknown.
```

Example

```
>>> from mc_libs_index.index import list_functions
>>> print(list_functions("mc_data_utils"))
```

7.2.5 Function `list_examples`

Return example calls for a library.

Syntax

```
from mc_libs_index.index import list_examples
list_examples(library: str) -> Dict[str, str]
```

Inputs

```
library (str): library name, e.g. ``"mc_data_utils"``.
```

Outputs

```
Dict[str, str]: mapping from fully qualified names to example strings.
```

Raises

```
KeyError: If the library is unknown.
```

Example

```
>>> from mc_libs_index.index import list_examples
>>> examples = list_examples("mc_data_utils")
```

7.2.6 Function `list_all`

Return the auto-discovered registry as a tree-like string.

Syntax

```
from mc_libs_index.index import list_all
list_all() -> str
```

Inputs

```
None
```

Outputs

```
str: tree-style list of all libraries and functions.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import list_all
>>> print(list_all())
```

7.2.7 Function `list_all_auto`

Return an auto-discovered registry by introspection.

Syntax

```
from mc_libs_index.index import list_all_auto
list_all_auto(base_path: str | Path | None = None) -> Dict[str, List[dict]]
```

Inputs

```
base_path (str | Path | None): base directory to scan (defaults to project root).
```

Outputs

```
Dict[str, List[dict]]: registry mapping for all libraries.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import list_all_auto
>>> registry = list_all_auto()
```

7.2.8 Function `get_help`

Return the docstring for a fully qualified function/class path.

Syntax

```
from mc_libs_index.index import get_help
get_help(func_path: str) -> str
```

Inputs

```
func_path (str): fully qualified name, e.g. ``"mc_data_utils.allan2"``.
```

Outputs

```
str: docstring text (or a fallback message).
```

Raises

```
ValueError: If ``func_path`` is not fully qualified.
```

Example

```
>>> from mc_libs_index.index import get_help
>>> print(get_help("mc_data_utils.allan2"))
```

7.2.9 Function `get_example`

Return the example block for a fully qualified function/class path.

Syntax

```
from mc_libs_index.index import get_example
get_example(func_path: str) -> str
```

Inputs

```
func_path (str): fully qualified name, e.g. ``"mc_data_utils.allan2"``.
```

Outputs

```
str: example text if available, else a fallback message.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import get_example
>>> print(get_example("mc_data_utils.allan2"))
```

7.2.10 Function `tree`

Return a tree-like string grouped by library and module.

Syntax

```
from mc_libs_index.index import tree
tree(libraries: Dict[str, List[dict]] | None = None) -> str
```

Inputs

```
libraries (Dict[str, List[dict]] | None): optional registry; when None, build one.
```

Outputs

```
str: tree-style summary of libraries and functions.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import tree
>>> print(tree())
```

7.2.11 Function `print_tree`

Print a tree-like list of libraries and function names.

Syntax

```
from mc_libs_index.index import print_tree
print_tree(libraries: Dict[str, List[dict]] | None = None) -> None
```

Inputs

```
libraries (Dict[str, List[dict]] | None): optional registry; when None, build one.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import print_tree
>>> print_tree()
```

7.2.12 Function `list_cli_commands`

Return a tree-like list of mc-libs CLI commands, usage, and targets.

Syntax

```
from mc_libs_index.index import list_cli_commands
list_cli_commands() -> str
```

Inputs

```
None
```

Outputs

```
str: tree-style list of CLI commands.
```

Raises Not specified.

Example

```
>>> from mc_libs_index.index import list_cli_commands
>>> print(list_cli_commands())
```

8 Package mc_reporting_utils

8.1 Module mc_reporting_utils

Human-facing reporting and formatting helpers.

No public functions or classes were found in this module.

8.2 Module mc_reporting_utils.formatting

Formatting utilities for human-readable reporting.

8.2.1 Function format_bytes_human

Format a byte count into a human-readable binary unit string.

Syntax

```
from mc_reporting_utils.formatting import format_bytes_human
format_bytes_human(num_bytes: int | float, precision: int = 2) -> str
```

Inputs

```
num_bytes (int | float): Size in bytes.
precision (int): Number of decimal places for non-byte units.
```

Outputs

```
str: Human-readable size, e.g. ``"254.24␣MB"``.
```

Raises Not specified.

Example

```
>>> format_bytes_human(512)
'512␣B'
>>> format_bytes_human(266591626)
'254.24␣MB'
```

8.2.2 Function format_timestamp_utc_us

Format a Unix timestamp in microseconds as UTC text.

Syntax

```
from mc_reporting_utils.formatting import format_timestamp_utc_us
format_timestamp_utc_us(timestamp_us: int | float | None) -> str
```

Inputs

```
timestamp_us (int | float | None): Unix timestamp in microseconds.
```

Outputs

```
str: formatted UTC string (``YYYY-mm-dd HH:MM:SS.ffffff UTC``) or
``"n/a"`` for invalid or non-positive values.
```

Raises Not specified.

Example

```
>>> format_timestamp_utc_us(1_700_000_000_000_000)
'2023-11-14␣22:13:20.000000␣UTC'
```

8.2.3 Function `format_duration_us`

Format a duration in microseconds as seconds text.

Syntax

```
from mc_reporting_utils.formatting import format_duration_us
format_duration_us(duration_us: int | float | None, precision: int = 3) -> str
```

Inputs

```
duration_us (int | float | None): duration in microseconds.
precision (int): number of decimal places in seconds.
```

Outputs

```
str: duration string in seconds (for example ``"0.125s"``). Returns
``"0.000s"`` for invalid or non-positive values.
```

Raises Not specified.

Example

```
>>> format_duration_us(125_000)
'0.125s'
```

8.2.4 Function `format_mean_ms`

Compute and format a mean duration in milliseconds.

Syntax

```
from mc_reporting_utils.formatting import format_mean_ms
format_mean_ms(total_us: int | float, samples: int) -> str
```

Inputs

```
total_us (int | float): accumulated duration in microseconds.
samples (int): number of samples in the accumulation.
```

Outputs

```
str: mean duration formatted in milliseconds (for example ``"3.4␣ms"``),
or ``"n/a"`` when ``samples <= 0`` or ``total_us`` is invalid.
```

Raises Not specified.

Example

```
>>> format_mean_ms(12_500, 5)
'2.5␣ms'
```

8.3 Module `mc_reporting_utils.terminal`

Terminal interaction helpers for human-facing reporting.

8.3.1 Class `Spinner`

Minimal terminal spinner for long-running operations.

The spinner animates only when the output stream is a TTY. In non-interactive runs it falls back to simple start/done text output.

Method `start` Start spinner animation and return the spinner instance.

Syntax

```
start(self) -> "Spinner"
```

Inputs

```
None
```

Outputs

```
Spinner: the same spinner object (useful for chaining/context use).
```

Raises Not specified.

Example

```
>>> from mc_reporting_utils.terminal import Spinner
>>> sp = Spinner("Loading").start()
>>> sp.stop("Done")
```

Method `stop` Stop spinner animation and optionally print a final message.

Syntax

```
stop(self, done_message: str | None = None) -> None
```

Inputs

```
done_message (str | None): message to print after stopping. If
``None``, uses the instance default ``done_message``.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from mc_reporting_utils.terminal import Spinner
>>> sp = Spinner("Working", done_message="Finished").start()
>>> sp.stop()
```

9 Package `mc_robo_utils`

9.1 Module `mc_robo_utils`

Utility helpers for robotics workflows.

Exports: - quat_to_rpy - KinematicsData, KinematicFrame, KinematicComponent, build_component, default_units - McKinematicsData, McKinematicFrame, McKinematicComponent (aliases) - plot_component, plot_multiple, save_html

No public functions or classes were found in this module.

9.2 Module `mc_robo_utils.kinematics`

Kinematics data structures for robotics workflows.

Classes: - KinematicComponent - KinematicFrame - KinematicsData - McKinematicComponent (alias) - McKinematicFrame (alias) - McKinematicsData (alias)

Functions: - build_component(...) - default_units()

9.2.1 Function `build_component`

Convenience constructor for "KinematicComponent".

Syntax

```
from mc_robo_utils.kinematics import build_component
build_component(
    x: Optional[np.ndarray] = None,
    y: Optional[np.ndarray] = None,
    z: Optional[np.ndarray] = None,
    psi: Optional[np.ndarray] = None,
    theta: Optional[np.ndarray] = None,
    phi: Optional[np.ndarray] = None,
) -> KinematicComponent
```

Inputs

```
x (Optional[np.ndarray]): x samples.
y (Optional[np.ndarray]): y samples.
z (Optional[np.ndarray]): z samples.
psi (Optional[np.ndarray]): yaw samples.
theta (Optional[np.ndarray]): pitch samples.
phi (Optional[np.ndarray]): roll samples.
```

Outputs

```
KinematicComponent: populated component container.
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.kinematics import build_component
>>> comp = build_component(x=[0.0, 1.0], y=[0.0, 2.0])
```

9.2.2 Function `default_units`

Default units for common kinematics fields.

Syntax

```
from mc_robo_utils.kinematics import default_units
default_units() -> Dict[str, str]
```

Inputs

```
None
```

Outputs

```
Dict[str, str]: mapping of kinematics field names to unit strings.
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.kinematics import default_units
>>> units = default_units()
>>> units["pos.x"]
'm'
```

9.2.3 Class `KinematicComponent`

Container for x,y,z and optional attitude angles psi, theta, phi.

Method `as_dict` Return the component fields as a dictionary.

Syntax

```
as_dict(self) -> dict
```

Inputs

```
None
```

Outputs

```
dict: mapping with keys ``x``, ``y``, ``z``, ``psi``, ``theta``, ``phi``.
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.kinematics import KinematicComponent
>>> comp = KinematicComponent(x=[0.0, 1.0], y=[2.0, 3.0])
>>> comp.as_dict()["x"]
[0.0, 1.0]
```

9.2.4 Class `KinematicFrame`

Container for pos, vel, acc groups in a frame (e.g., NED or body).

No public methods were found.

9.2.5 Class `KinematicsData`

Standard kinematics structure shared across projects.

Method `copy` Return a shallow copy of the kinematics data.

Syntax

```
copy(self) -> "KinematicsData"
```

Inputs

```
None
```

Outputs

```
KinematicsData: new instance with copied time arrays and same frame objects.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_robo_utils.kinematics import KinematicsData
>>> data = KinematicsData(t_s=np.array([0.0, 1.0]))
>>> data_copy = data.copy()
```

9.3 Module `mc_robo_utils.plotting`

Plotting helpers for kinematics data using Plotly.

Functions: - plot_component(...) - plot_multiple(...) - plot_series(x, y, title=None, x_label="time", y_label=None, height=400) - plot_multi_series(series, title=None) - plot_allan(T_g, sigma_g, T_a, sigma_a, labels=None, colors=None) - save_html(fig, path, open_after=False)

9.3.1 Function `plot_component`

Plot a single component (pos/vel/acc) from a frame.

Syntax

```
from mc_robo_utils.plotting import plot_component
plot_component(
    data: McKinematicsData,
    frame: str = "ned",
    group: str = "pos",
    fields: Optional[Iterable[str]] = None,
    separate_axes: bool = True,
    title: Optional[str] = None,
) -> go.Figure
```

Inputs

```
data (McKinematicsData): kinematics data structure.
frame (str): frame name (``"ned"`` or ``"body"``).
group (str): component group (``"pos"``, ``"vel"``, or ``"acc"``).
fields (Optional[Iterable[str]]): subset of ``[x, y, z, psi, theta, phi]`` to plot.
separate_axes (bool): when True, one subplot per field.
title (Optional[str]): optional figure title.
```

Outputs

```
go.Figure: Plotly figure containing the plot(s).
```

Raises

```
ValueError: If the frame/group is invalid or no fields are available.
```

Example

```
>>> from mc_robo_utils.plotting import plot_component
>>> fig = plot_component(data, frame="body", group="acc", fields=["x", "y", "z"])
```

9.3.2 Function `plot_multiple`

Plot multiple components in a single figure.

Syntax

```
from mc_robo_utils.plotting import plot_multiple
plot_multiple(
    data: McKinematicsData,
    specs: Iterable[tuple[str, str, Iterable[str]]],
    separate_axes: bool = True,
    title: Optional[str] = None,
) -> go.Figure
```

Inputs

```
data (McKinematicsData): kinematics data structure.
specs (Iterable[tuple[str, str, Iterable[str]]]): list of (frame, group, fields).
separate_axes (bool): when True, one subplot per field.
title (Optional[str]): optional figure title.
```

Outputs

```
go.Figure: Plotly figure containing the plot(s).
```

Raises

```
ValueError: If ``specs`` is empty.
```

Example

```
>>> from mc_robo_utils.plotting import plot_multiple
>>> fig = plot_multiple(data, specs=[("body", "vel", ["x", "y", "z"])])
```

9.3.3 Function `plot_series`

Plot a single series (x vs y).

Syntax

```
from mc_robo_utils.plotting import plot_series
plot_series(
    x: Sequence | np.ndarray,
    y: Sequence | np.ndarray,
    *,
    title: str | None = None,
    x_label: str = "time",
    y_label: str | None = None,
    height: int = 400,
) -> go.Figure
```

Inputs

```
x (Sequence | np.ndarray): x values.
y (Sequence | np.ndarray): y values.
title (str | None): optional title.
x_label (str): x-axis label.
y_label (str | None): y-axis label.
height (int): figure height in pixels.
```

Outputs

```
go.Figure: Plotly figure containing the series.
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.plotting import plot_series
>>> fig = plot_series([0, 1, 2], [0, 1, 0], title="demo")
```

9.3.4 Function `plot_multi_series`

Plot multiple labeled series as stacked Plotly subplots.

Syntax

```
from mc_robo_utils.plotting import plot_multi_series
plot_multi_series(
    series: Iterable[tuple[Sequence[Any], ArrayLike, str, str | None]],
    *,
    title: str | None = None,
    height_per: int = 250,
) -> go.Figure
```

Inputs

```
series (Iterable[tuple[Sequence[Any], ArrayLike, str, str | None]]):
    iterable of ``(x, y, label, y_label)`` tuples.
title (str | None): optional figure title.
height_per (int): height in pixels allocated per subplot.
```

Outputs

```
go.Figure: Plotly figure containing one subplot per provided series.
```

Raises

```
ValueError: If ``series`` is empty.
```

Example

```
>>> from mc_robo_utils.plotting import plot_multi_series
>>> fig = plot_multi_series([
... ([0, 1, 2], [0.0, 1.0, 0.0], "signal_a", "amp"),
... ([0, 1, 2], [2.0, 2.5, 2.3], "signal_b", "deg"),
... ])
```

9.3.5 Function `plot_allan`

Plot gyro/accel Allan deviation with default markers/labels.

Syntax

```
from mc_robo_utils.plotting import plot_allan
plot_allan(
    T_g: np.ndarray,
    sigma_g: np.ndarray,
    T_a: np.ndarray,
    sigma_a: np.ndarray,
    *,
    labels: Optional[Iterable[str]] = None,
    colors: Optional[Iterable[str]] = None,
    title: str = "Allan␣Deviation",
) -> go.Figure
```

Inputs

```
T_g (np.ndarray): gyro tau array.
sigma_g (np.ndarray): gyro Allan deviation array (K x 3).
T_a (np.ndarray): accel tau array.
sigma_a (np.ndarray): accel Allan deviation array (K x 3).
labels (Optional[Iterable[str]]): axis labels (default ``["X", "Y", "Z"]``).
colors (Optional[Iterable[str]]): trace colors.
title (str): figure title.
```

Outputs

```
go.Figure: Plotly figure with two stacked log-log plots.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_robo_utils.plotting import plot_allan
>>> T = np.logspace(0, 3, 10)
>>> sigma = np.random.rand(10, 3)
>>> fig = plot_allan(T, sigma, T, sigma)
```

9.3.6 Function `save_html`

Save a Plotly figure to HTML, optionally opening it.

Syntax

```
from mc_robo_utils.plotting import save_html
save_html(fig: go.Figure, path: str, open_after: bool = False) -> None
```

Inputs

```
fig (go.Figure): Plotly figure to save.
path (str): destination HTML path.
open_after (bool): when True, open the file after writing.
```

Outputs

```
None
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.plotting import plot_series, save_html
>>> fig = plot_series([0, 1], [0, 1])
>>> save_html(fig, "plot.html", open_after=False)
```

9.4 Module `mc_robo_utils.plotting_print`

Matplotlib plotting helpers for kinematics and diagnostics (print/static).

Functions: - plot_multiple_mpl(data, specs, t_dt=None, title=None) - plot_series_mpl(x, y, title=None, x_label=None, y_label=None) - plot_multi_series_mpl(series, title=None, x_label="time") - plot_allan_mpl(T_g, sigma_g, T_a, sigma_a)

9.4.1 Function `plot_multiple_mpl`

Plot multiple components using Matplotlib (one subplot per field).

Syntax

```
from mc_robo_utils.plotting_print import plot_multiple_mpl
plot_multiple_mpl(
    data: McKinematicsData,
    specs: Iterable[tuple[str, str, Iterable[str]]],
    *,
    t_dt: Optional[Sequence] = None,
    title: Optional[str] = None,
)
```

Inputs

```
data (McKinematicsData): kinematics data to plot.
specs (Iterable[tuple[str, str, Iterable[str]]]): list of (frame, group, fields).
t_dt (Optional[Sequence]): optional datetime array for the x-axis.
title (Optional[str]): optional figure title.
```

Outputs

```
matplotlib.figure.Figure: Matplotlib figure containing the plots.
```

Raises

```
ValueError: If ``specs`` is empty.
```

Example

```
>>> from mc_robo_utils.plotting_print import plot_multiple_mpl
>>> fig = plot_multiple_mpl(data, specs=[("body", "vel", ["x", "y"])])
```

9.4.2 Function `plot_series_mpl`

Plot a single series using Matplotlib.

Syntax

```
from mc_robo_utils.plotting_print import plot_series_mpl
plot_series_mpl(x, y, *, title=None, x_label=None, y_label=None)
```

Inputs

```
x: x-axis values.
y: y-axis values.
title: optional title.
x_label: optional x-axis label.
y_label: optional y-axis label.
```

Outputs

```
matplotlib.figure.Figure: Matplotlib figure containing the series.
```

Raises Not specified.

Example

```
>>> from mc_robo_utils.plotting_print import plot_series_mpl
>>> fig = plot_series_mpl([0, 1], [0, 1], title="demo")
```

9.4.3 Function `plot_multi_series_mpl`

Plot multiple labeled series as stacked Matplotlib subplots.

Syntax

```
from mc_robo_utils.plotting_print import plot_multi_series_mpl
plot_multi_series_mpl(series, *, title=None, x_label="time")
```

Inputs

```
series: iterable of ``(x, y, label, y_label)`` tuples.
title: optional figure title.
x_label: x-axis label for the bottom subplot.
```

Outputs

```
matplotlib.figure.Figure: Matplotlib figure containing one subplot per
input series.
```

Raises

```
ValueError: If ``series`` is empty.
```

Example

```
>>> from mc_robo_utils.plotting_print import plot_multi_series_mpl
>>> fig = plot_multi_series_mpl([
... ([0, 1], [0.0, 1.0], "speed", "m/s"),
... ([0, 1], [1.0, 1.5], "yaw", "deg"),
... ])
```

9.4.4 Function `plot_allan_mpl`

Plot gyro/accel Allan deviation using Matplotlib.

Syntax

```
from mc_robo_utils.plotting_print import plot_allan_mpl
plot_allan_mpl(T_g: np.ndarray, sigma_g: np.ndarray, T_a: np.ndarray, sigma_a: np.ndarray
    )
```

Inputs

```
T_g (np.ndarray): gyro tau array.
sigma_g (np.ndarray): gyro Allan deviation array (K x 3).
T_a (np.ndarray): accel tau array.
sigma_a (np.ndarray): accel Allan deviation array (K x 3).
```

Outputs

```
matplotlib.figure.Figure: Matplotlib figure containing the Allan plots.
```

Raises Not specified.

Example

```
>>> import numpy as np
>>> from mc_robo_utils.plotting_print import plot_allan_mpl
>>> T = np.logspace(0, 3, 10)
>>> sigma = np.random.rand(10, 3)
>>> fig = plot_allan_mpl(T, sigma, T, sigma)
```

9.5 Module `mc_robo_utils.transformations`

Common robotics transformations.

Functions: - quat_to_rpy(quat_xyzw)

9.5.1 Function `quat_to_rpy`

Convert quaternion(s) (x, y, z, w) to roll/pitch/yaw (rad).

Syntax

```
from mc_robo_utils.transformations import quat_to_rpy
quat_to_rpy(quat_xyzw: np.ndarray) -> np.ndarray
```

Inputs

```
quat_xyzw: Array of shape (N, 4) or (4,) in x,y,z,w order.
```

Outputs

```
Array of shape (N, 3) with roll, pitch, yaw in radians.
```

Raises

```
ValueError: If the input array does not have shape ``(4,)`` or ``(N, 4)``.
```

Example

```
>>> import numpy as np
>>> from mc_robo_utils.transformations import quat_to_rpy
>>> quat_to_rpy(np.array([0.0, 0.0, 0.0, 1.0]))
```